2023

Q. Describe design concept of modularity and separation of concerns in your own words. Is there a case when a "divide and conquer" strategy may not be appropriate? How might such a case affect the argument for modularity?

Separation of concerns is the practice of breaking a software system down into separate parts, each of which focuses on a specific concern. This helps to ensure that software systems are easy to understand, maintain, and modify. There may be cases when a "divide and conquer" strategy is not appropriate, such as when the parts of the system are tightly coupled and cannot be easily separated. In such cases, the argument for modularity is weakened.

Separation of concerns:

Separation of Concerns is a design concept. It refers to the delineation and correlation of software elements to achieve order within a system. That means a large and difficult problem can be divided into small parts and can be done easily. By dividing the work into parts and handling it, the effort decreases. By this we can handle complex problem more easily through proper separation of concerns. So complexity becomes manageable.

Yes, there is a case when a divide-and-conquer strategy may not be appropriate. The strategy may not be appropriate in the case where, the complexity when two problems are combined is greater than the sum of perceived complexity when each is taken separately. The argument for modularity is that, if you subdivide software indefinitely the effort required to develop it will become negligibly small. But in these types of cases, as the number of modules grows, the effort (cost) associated with integrating the modules grow. Hence the argument for modularity will not be appropriate.

Design Concept of Modularity and Separation of Concerns

Modularity:

- Modularity is the design principle of breaking down a system into smaller, self-contained units or modules. Each module focuses on a specific part of the system's functionality and can be developed, tested, and maintained independently. This approach enhances code readability, reusability, and maintainability. Modules often interact through well-defined interfaces.

Separation of Concerns:

- Separation of concerns is a principle that involves dividing a program into distinct sections, where each section addresses a separate concern or aspect of the program's functionality. This helps in isolating changes and understanding the code more easily. For instance, separating data handling from the user interface logic or separating business logic from presentation logic.

Case When "Divide and Conquer" Strategy May Not Be Appropriate

A "divide and conquer" strategy, which involves breaking down a problem into smaller, more manageable parts, may not be appropriate in scenarios where the problem inherently requires a holistic approach. Examples include:

1. **Complex Interdependencies**: If the components of a system are highly interdependent, breaking them into separate modules can lead to complications in managing these dependencies. For example, in tightly coupled systems where changes in one part of the system may have cascading effects on other parts, dividing the system can introduce significant integration challenges.
2. **Performance Constraints**: In cases where performance is a critical concern, dividing a system into smaller modules may introduce overheads related to communication and synchronization between modules. Real-time systems or high-frequency trading platforms may require tightly integrated designs to meet stringent performance requirements.

3. **Simplicity and Overhead**: For very simple or small-scale problems, the overhead of dividing the system into modules may not be justified. The added complexity of managing multiple modules can outweigh the benefits, leading to inefficient use of resources.

Impact on the Argument for Modularity

While modularity is generally a strong design principle, its applicability needs to be evaluated based on the specific context of the project. In cases where modularity introduces more complexity or overhead than it resolves, the argument for modularity weakens. It's important to balance the benefits of modularity with the practical aspects of the problem at hand. Factors such as performance requirements, system complexity, and the nature of dependencies should guide the decision on whether to apply modularity and to what extent.

Q. Using your own words, describe the difference between verification and validation.
Do both make use of test-case design methods and testing strategies?

Following are the differences between verification and validation:
Verification: Verification refers to a set of tasks that are carried out to check whether the software correctly implements a specific function.
Verification focuses on process.
It checks whether the product that is built is correct or not.
The objective of verification is to ensure that the product meets the requirements and design specifications.
It deals with internal product specifications.
Validation: Validation refers to a different set of activities that are carried out to ensure that the software is built as per the customer requirements.
Validation focuses on product.
It checks whether the right product is built or not.
The objective of validation is to ensure that the product meets the user requirements.
It deals directly with users and their requirements.

Both Verification and Validation make use of test case design methods and testing strategies. The testing strategy is based on the spiral model. A test plan specifies the tests to be conducted and the process of testing to uncover any errors in the product. A test case consists of a test data to be given along with the expected output.

Verification and validation are both critical aspects of ensuring software quality, but they focus on different goals:

- **Verification** is about checking if the software meets the specified requirements and design specifications. It's a process of evaluating the intermediate work products of a software development lifecycle to ensure they are correct and consistent. The main question verification answers is, "Are we building the product right?"
- **Validation** is about evaluating the final product to ensure it meets the business needs and requirements. It involves checking that the software performs as expected in the real-world scenario and satisfies the intended use. The main question validation answers is, "Are we building the right product?"

Both verification and validation make use of test-case design methods and testing strategies, but they apply them in different contexts.

- **Verification** uses techniques like inspections, reviews, walkthroughs, and static analysis to ensure the design and implementation meet the requirements. It often involves testing at various stages, including unit testing and integration testing, to verify that individual components and their integrations are correct.
- **Validation** uses techniques such as system testing, acceptance testing, and user testing to ensure the software meets user needs and requirements. It often involves running the software in a production-like environment to validate its behavior under real-world conditions.

In summary, verification focuses on the correctness of the development process, while validation focuses on the correctness of the final product. Both processes use testing strategies to achieve their goals.

Verification	Validation
Verification refers to a set of tasks that are carried out to check whether the software correctly implements a specific function.	Validation refers to a different set of activities that are carried out to ensure that the software is built as per the customer requirements.
Verification focuses on process.	Validation focuses on product.
It checks whether the product that is built is correct or not.	It checks whether the right product is built or not.
The objective of verification is to ensure that the product meets the requirements and design specifications.	The objective of validation is to ensure that the product meets the user requirements.
It deals with internal product specifications.	It deals directly with users and their requirements.
The techniques of the verification are reviews, meetings, inspections etc.	The techniques of the validation are white box testing, black box testing, gray box testing etc.
The quality assurance (QA) team does verification.	The testing team does validation.
Verification is done before preforming validation	Validation is done after preforming verification

Q. Interpret different elements of analysis model using different digrams.

Creating a diagram to interpret different elements of an analysis model typically involves visualizing various components and their relationships. Here's a breakdown of the common elements of an analysis model using a diagram:

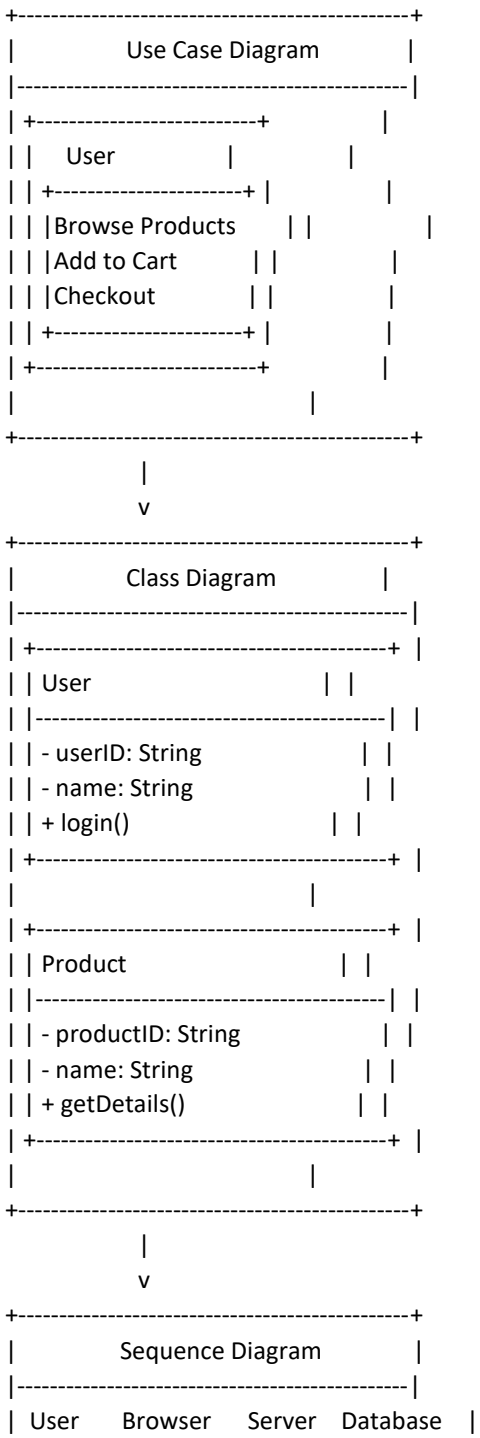
Analysis Model Elements

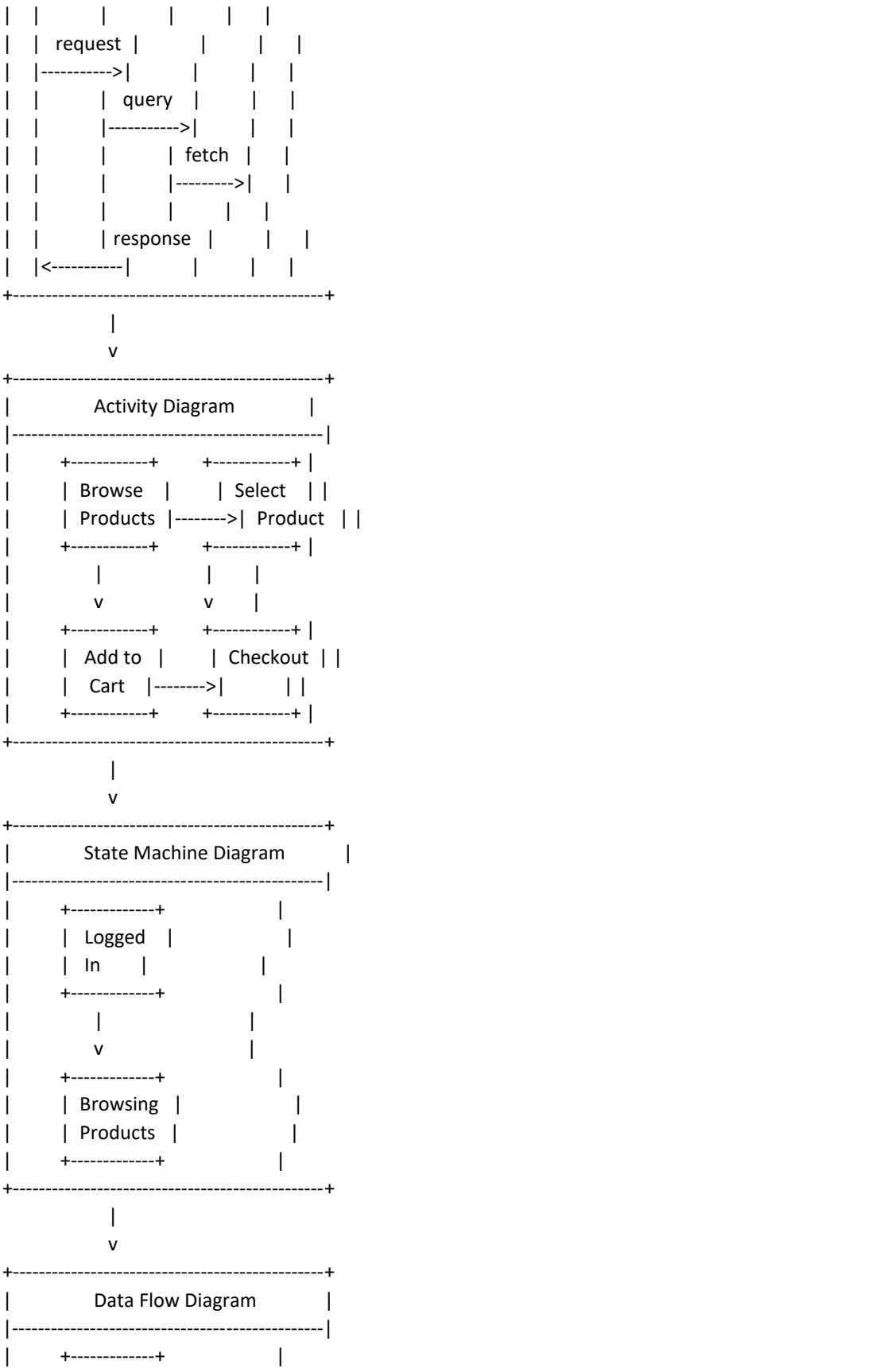
1. *Use Case Diagram*
2. *Class Diagram*
3. *Sequence Diagram*
4. *Activity Diagram*
5. *State Machine Diagram*

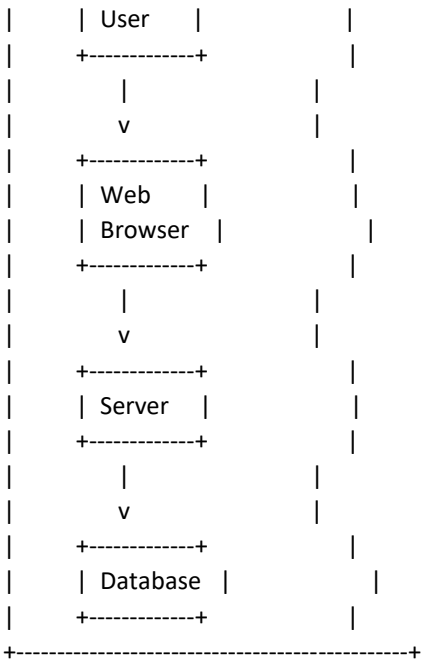
6. *Data Flow Diagram (DFD)*

Example Diagram

Below is a textual representation of how these elements might be arranged in a diagram for a generic e-commerce system:




request
query
fetch
response
Activity Diagram
Browse Products
Select Product
Add to Cart
Checkout
State Machine Diagram
Logged In
Browsing Products
Data Flow Diagram



Descriptions

1. *Use Case Diagram*: Represents the functional requirements of the system. It shows interactions between users (actors) and the system through use cases (e.g., "Browse Products", "Checkout").

2. *Class Diagram*: Depicts the structure of the system by showing its classes, attributes, operations, and the relationships among objects. For example, classes like User and Product with their properties and methods.

3. *Sequence Diagram*: Illustrates how objects interact in a particular scenario of a use case. It shows the sequence of messages exchanged between objects (e.g., between User, Browser, Server, and Database).

4. *Activity Diagram*: Represents the workflow of a particular business process or use case. It shows the flow of activities and decisions (e.g., browsing products, adding to cart, and checking out).

5. *State Machine Diagram*: Describes the states an object can be in and how it transitions from one state to another based on events (e.g., Logged In, Browsing Products).

6. *Data Flow Diagram (DFD)*: Represents the flow of data within the system. It shows how data moves between processes (e.g., from User to Web Browser, Server, and Database).

These diagrams collectively provide a comprehensive view of the system from different perspectives, facilitating better understanding, communication, and design of the software.

Q. Can a program be correct and still not exhibit a good quality?

Yes, it is possible for a program to conform to all explicit functional and performance requirements at a given instant, yet to not perform well during maintenance, integration on other systems, or might have hardware dependencies.

Many programs have been patched together, so that they work yet these programs exhibit very low quality if measured by McCall's quality factors.

Yes: the interface can be inconvenient, some secondary features may not work properly. An example is SnowRunner game: it works well in single player mode with default vehicles, but most users experience trouble connecting to a multiplayer session and trying to use mods.

Yes, of course. If the definition of what is wanted and what the inputs are is not specified adequately/correctly then any program written will be inadequate or just plain bad.

Q. Considering each of the four aspects of the cost of quality, conclude which do you think is the most expensive and why? Also list Gravin's quality dimensions.

The four aspects of the cost of quality are Prevention Costs, Appraisal Costs, Internal Failure Costs, and External Failure Costs. Let's briefly discuss each aspect before addressing the most expensive one and Gravin's quality dimensions.

Cost of Quality Aspects

1. **Prevention Costs**:
    - **Description**: Costs incurred to prevent defects from occurring in the first place, such as training, quality planning, process improvement, and proper documentation.
    - **Purpose**: Reduce the likelihood of defects, thereby minimizing rework and failure costs downstream.
2. **Appraisal Costs**:
    - **Description**: Costs associated with evaluating and assessing products or services to ensure they meet quality standards, including inspection, testing, and quality audits.
    - **Purpose**: Identify defects early in the process to prevent them from reaching customers and causing external failure costs.
3. **Internal Failure Costs**:
    - **Description**: Costs incurred due to defects found before the product is delivered to the customer, such as rework, scrap, and downtime.
    - **Purpose**: Address and correct defects internally to prevent them from leading to more costly external failures.
4. **External Failure Costs**:
    - **Description**: Costs resulting from defects found by customers after delivery, such as warranty claims, recalls, customer support, and lost sales or reputation.
    - **Purpose**: Minimize the impact of defects that reach customers, which can be significantly more expensive to resolve compared to internal failures.

Most Expensive Aspect of the Cost of Quality

Among the four aspects, **Appraisal Costs** can often be the most expensive. Here's why this is the case:

- **Intensive Testing**: Appraisal costs involve rigorous testing and inspection processes to identify defects before they escalate further downstream. This includes comprehensive quality audits, testing cycles, and thorough inspection procedures.
- **Resource Intensive**: Appraisal processes require dedicated resources such as skilled personnel, testing equipment, and specialized software tools. The cumulative cost of these resources can be substantial, particularly in large-scale projects or complex systems.
- **Early Detection**: While prevention costs aim to eliminate defects proactively and internal failure costs address issues found during development, appraisal costs focus on early detection through meticulous testing. Detecting and fixing defects at this stage can prevent costly repercussions in later stages, such as external failure costs.

Most Expensive Aspect of the Cost of Quality

Among the four aspects, **External Failure Costs** are typically the most expensive. Here's why:

- **Impact**: Defects that reach customers can lead to significant financial losses, including warranty claims, product recalls, legal liabilities, and damage to the company's reputation.
- **Costs**: Resolving issues after delivery often requires extensive resources for customer support, replacement or repair of products, and potential financial penalties.
- **Long-term Effects**: Negative publicity and loss of customer trust can have lasting effects on sales and market share, affecting the company's revenue and growth.

Gravin's Quality Dimensions

Gravin's quality dimensions provide a comprehensive framework to assess and ensure product or service quality. They include:

1. **Performance Quality**: The primary operating characteristics of a product or service, such as speed, accuracy, and reliability.
2. **Feature Quality**: Additional characteristics that enhance a product's appeal and usability, such as additional functionalities or capabilities.
3. **Reliability**: The ability of a product or service to perform consistently and predictably over time under normal conditions.
4. **Conformance**: The degree to which a product or service meets established standards, specifications, or regulations.
5. **Durability**: The lifespan and resilience of a product or service, including its ability to withstand wear and tear.
6. **Serviceability**: The ease and efficiency with which a product can be maintained, serviced, or repaired.
7. **Aesthetics**: The visual appeal, design, and sensory attributes of a product or service.
8. **Perception**: Customer perceptions and expectations regarding quality, brand reputation, and user satisfaction.

Conclusion

In conclusion, while all aspects of the cost of quality are important, External Failure Costs tend to be the most expensive due to their direct impact on customer satisfaction, brand reputation, and financial losses. Addressing quality issues early through effective prevention, appraisal, and internal controls can help mitigate the risk of incurring these costly external failures. Gravin's quality dimensions provide a holistic framework to ensure that products or services meet not only functional requirements but also customer expectations and industry standards across various dimensions of quality.

Q. Discuss how does agile communication differ from traditional software engineering communication and how is it similar? Why does an iterative process make it easier to manage change? Is every agile process is iterative? Is it possible to complete a project in just one iteration and still be agile? Explain your answers

Differences Between Agile and Traditional Software Engineering Communication

1. **Frequency and Nature of Communication**:
    - **Agile**: Emphasizes continuous communication with frequent, informal meetings such as daily stand-ups, sprint planning, and retrospectives. Communication is often face-to-face and involves all stakeholders, including customers.
    - **Traditional**: Relies more on formal, scheduled meetings at specific project milestones (e.g., requirements gathering, design reviews, status reports). Communication tends to be more document-driven and less frequent.
2. **Documentation**:
    - **Agile**: Focuses on working software over comprehensive documentation. Documentation is kept to a minimum and is often lightweight.
    - **Traditional**: Requires extensive documentation at every phase of the project (e.g., requirement specifications, design documents, test plans).

3. **Stakeholder Involvement**:
    - **Agile**: Encourages active and continuous involvement of stakeholders, ensuring feedback is integrated regularly.
    - **Traditional**: Stakeholders are typically involved at the beginning and end of the project or at major milestones.

Similarities Between Agile and Traditional Software Engineering Communication

1. **Goal-Oriented**: Both aim to ensure clear communication of project goals, progress, and any issues that arise.
2. **Team Collaboration**: Both methods value collaboration within the team, although the methods and frequency of collaboration differ.

Why an Iterative Process Makes it Easier to Manage Change

An iterative process makes it easier to manage change because:

- **Incremental Delivery**: Projects are broken down into smaller, manageable chunks that are delivered incrementally. This allows teams to quickly adapt to changes in requirements, technology, or market conditions.
- **Regular Feedback**: Frequent iterations allow for regular feedback from stakeholders, enabling teams to identify and address issues early.
- **Flexibility**: Each iteration builds on the previous one, providing the flexibility to refine and adjust features and functionality as the project progresses.
- **Risk Mitigation**: Early and continuous testing in each iteration helps identify and mitigate risks sooner.

Iterative Nature of Agile Processes

Not every agile process is strictly iterative, but most incorporate iterative elements. For example:

- **Scrum**: Iterative by design with its sprints.
- **Kanban**: Focuses more on continuous delivery rather than fixed iterations but can be adapted to include iterative cycles.

Completing a Project in One Iteration and Being Agile

It is theoretically possible to complete a project in one iteration and still be considered agile if:

- **Small Scope**: The project scope is small enough to be completed within one iteration.
- **Continuous Delivery**: The project still adheres to agile principles such as continuous delivery, customer collaboration, and flexibility.
- **Adaptability**: The team remains adaptable and open to changes, even if the project is delivered in one go.

However, this scenario is rare and not typical for most agile projects, which are designed to handle complexity and changing requirements over multiple iterations.

Q. Provide an example of software development project that word be amenable to prototyping. Name an application that would be more difficult to prototype.

Example of a Software Development Project Amenable to Prototyping

Project: E-commerce Website

Reason for Amenability to Prototyping:

1. **User Interface Focused**: E-commerce websites heavily rely on user-friendly interfaces. Prototyping allows for early feedback on design and usability from potential users.
2. **Clear Requirements**: Basic functionalities like product listings, shopping carts, and checkout processes are well understood, making it easier to create initial prototypes.
3. **Iterative Improvement**: Prototyping enables iterative improvements based on user feedback, ensuring the final product meets user expectations.
4. **Risk Reduction**: Early prototypes help identify potential issues with navigation, layout, and user experience, reducing risks in later stages.

Example Prototype Features:

- Homepage with product categories
- Product detail pages
- Shopping cart functionality
- Basic checkout process

Example of an Application More Difficult to Prototype
Application: Real-Time Financial Trading Platform
Reason for Difficulty in Prototyping:

1. **Complex Back-End Logic**: Real-time trading platforms require complex algorithms and real-time data processing, which are difficult to simulate in early prototypes.
2. **High Reliability and Security**: These platforms must be highly reliable and secure, with stringent requirements that are not easily captured in a prototype.
3. **Integration with External Systems**: They often need to integrate with various external financial systems and APIs, making it hard to create a fully functional prototype.
4. **Performance Requirements**: Real-time performance and low latency are critical, and early prototypes may not accurately reflect the final system's performance characteristics.

Example Prototype Challenges:

- Simulating real-time data feeds and market conditions
- Implementing and testing complex trading algorithms
- Ensuring data security and regulatory compliance in a prototype environment.

(imp) Q. Explore Software Requirement Engineering Tasks in detail.

Software Requirement Engineering (SRE) encompasses a series of tasks and processes aimed at understanding, documenting, and managing the requirements of a software system throughout its lifecycle. Each task plays a crucial role in ensuring that the software meets stakeholder expectations and business needs. Let's explore each task in detail:

Software Requirement Engineering Tasks

1. **Inception**:
    - **Description**: The inception phase marks the beginning of requirement engineering, where initial ideas and concepts for the software system are explored and defined.
    - **Activities**:
        - Identify stakeholders and their concerns.
        - Define the scope of the software project.
        - Outline high-level goals and objectives.
        - Perform feasibility analysis to assess technical, economic, and organizational feasibility.
2. **Elicitation**:
    - **Description**: Elicitation involves gathering and discovering requirements from stakeholders, users, and other sources to understand their needs and expectations.
    - **Activities**:
        - Conduct interviews, workshops, and brainstorming sessions with stakeholders.
        - Use techniques such as surveys, questionnaires, and observations to collect requirements.
        - Analyze existing documentation and systems to extract relevant information.
        - Identify non-functional requirements (e.g., performance, security) alongside functional requirements.
3. **Elaboration**:
    - **Description**: Elaboration refines and details the requirements gathered during elicitation to ensure clarity, completeness, and consistency.

- **Activities**:
  - Prioritize and categorize requirements based on importance and feasibility.
  - Create use cases, scenarios, and user stories to describe system behavior and interactions.
  - Specify detailed functional and non-functional requirements using appropriate formats and tools.
  - Validate requirements with stakeholders to confirm understanding and accuracy.

4. **Negotiation**:
   - **Description**: Negotiation involves reconciling conflicting requirements and priorities among stakeholders to achieve consensus and agreement.
   - **Activities**:
     - Facilitate discussions and meetings to resolve disagreements or conflicting priorities.
     - Seek compromise and trade-offs while balancing competing interests.
     - Document decisions and agreements reached during negotiation processes.
5. **Specification**:
   - **Description**: Specification involves documenting requirements in a structured format that is understandable to stakeholders, developers, and testers.
   - **Activities**:
     - Write clear, concise, and unambiguous requirement documents, including functional specifications, use cases, and system requirements specifications (SRS).
     - Use diagrams, charts, and models to enhance understanding and visualization of requirements.
     - Review and validate specifications with stakeholders to ensure alignment with their expectations.
6. **Validation**:
   - **Description**: Validation ensures that the documented requirements accurately reflect stakeholder needs and will lead to the development of a system that meets those needs.
   - **Activities**:
     - Perform reviews, inspections, and walkthroughs of requirement documents to identify errors, inconsistencies, and missing details.
     - Use prototyping and simulation techniques to validate requirements in a practical context.
     - Conduct acceptance testing and user acceptance testing (UAT) to confirm that the implemented system meets specified requirements.
7. **Requirement Management**:
   - **Description**: Requirement management involves managing changes to requirements throughout the software development lifecycle, ensuring traceability and alignment with project goals.
   - **Activities**:
     - Establish a baseline of approved requirements and manage changes through a formal change control process.
     - Track and document requirements traceability to ensure that each requirement is addressed by the design, implementation, and testing phases.
     - Communicate changes and updates to stakeholders and project team members.
     - Maintain a repository or database of requirements and associated documentation for reference and future audits.

Conclusion

Software Requirement Engineering tasks are essential for establishing a solid foundation for software development projects. By systematically performing inception, elicitation, elaboration, negotiation, specification, validation, and requirement management tasks, teams can effectively capture, analyze, document, and manage requirements to

deliver successful software systems that meet stakeholder expectations and business objectives. Each task contributes to reducing risks, improving communication, and ensuring the quality and completeness of software requirements throughout the project lifecycle.

Q. You have been assigned to a software team that is developing a hospital management system, it has entities like patient, doctors and receptionist,
a. Draw a use case diagram for proposed system from the perception of doctors (general doctor and specialist).
b. Define types of relationships used in class diagram provide a class diagram for the proposed system with all possible types of relationships of class diagram.

a. Use Case Diagram for Hospital Management System (Doctors' Perspective)
Below is the textual representation of the use case diagram from the perception of doctors (general doctor and specialist):

Actors:

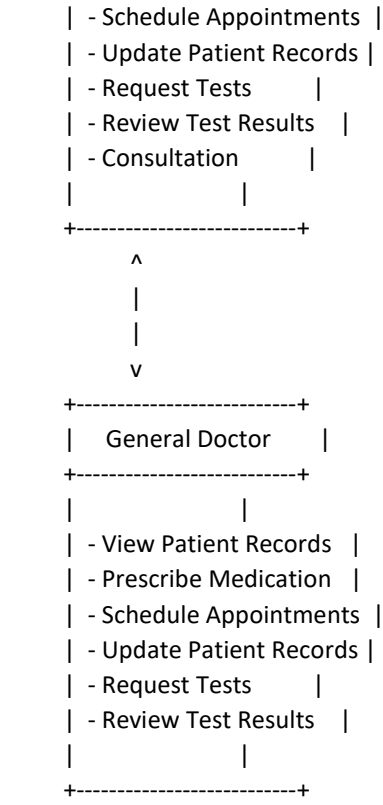
- General Doctor
- Specialist
- System (Hospital Management System)

Use Cases:

1. **View Patient Records**
    - Actor: General Doctor, Specialist
    - Description: Allows doctors to view the medical history and current health status of their patients.
2. **Prescribe Medication**
    - Actor: General Doctor, Specialist
    - Description: Enables doctors to prescribe medications to patients.
3. **Schedule Appointments**
    - Actor: General Doctor, Specialist
    - Description: Allows doctors to schedule follow-up appointments with patients.
4. **Update Patient Records**
    - Actor: General Doctor, Specialist
    - Description: Allows doctors to update patient information and medical records.
5. **Request Tests**
    - Actor: General Doctor, Specialist
    - Description: Enables doctors to request diagnostic tests for patients.
6. **Review Test Results**
    - Actor: General Doctor, Specialist
    - Description: Allows doctors to review the results of diagnostic tests performed on patients.
7. **Consultation**
    - Actor: Specialist
    - Description: Allows specialists to conduct specialized consultations for patients referred by general doctors.

plaintext
Copy code

```
+---------------------------+
|     Specialist        |
+---------------------------+
|                   |
| - View Patient Records   |
| - Prescribe Medication   |
```



b. Types of Relationships in Class Diagrams

1. **Association**: A relationship between two classes that establishes a connection where instances of one class can be connected to instances of another class.
    - Example: A Doctor class is associated with a Patient class.
2. **Aggregation**: A special form of association that represents a whole-part relationship. The part can exist independently of the whole.
    - Example: A Hospital class aggregates multiple Department classes.
3. **Composition**: A stronger form of aggregation where the part cannot exist without the whole.
    - Example: A Patient class composes a MedicalRecord class.
4. **Inheritance (Generalization)**: A relationship where one class (child) inherits attributes and methods from another class (parent).
    - Example: A Specialist class inherits from a Doctor class.
5. **Dependency**: A relationship where one class depends on another class. A change in one class might affect the other.
    - Example: A Prescription class depends on both Doctor and Patient classes.

Class Diagram for Hospital Management System

Here is the textual representation of a class diagram with all possible types of relationships:

Classes:

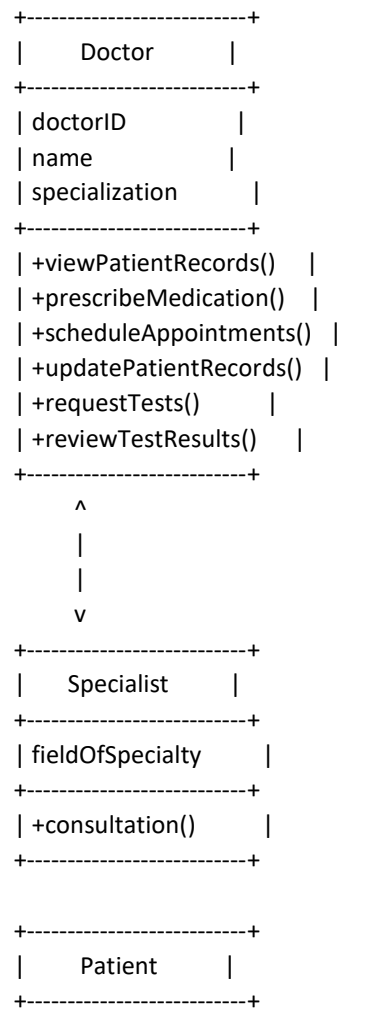
1. Doctor
    - Attributes: doctorID, name, specialization
    - Methods: viewPatientRecords(), prescribeMedication(), scheduleAppointments(), updatePatientRecords(), requestTests(), reviewTestResults()
2. Specialist (inherits from Doctor)
    - Attributes: fieldOfSpecialty
    - Methods: consultation()
3. Patient

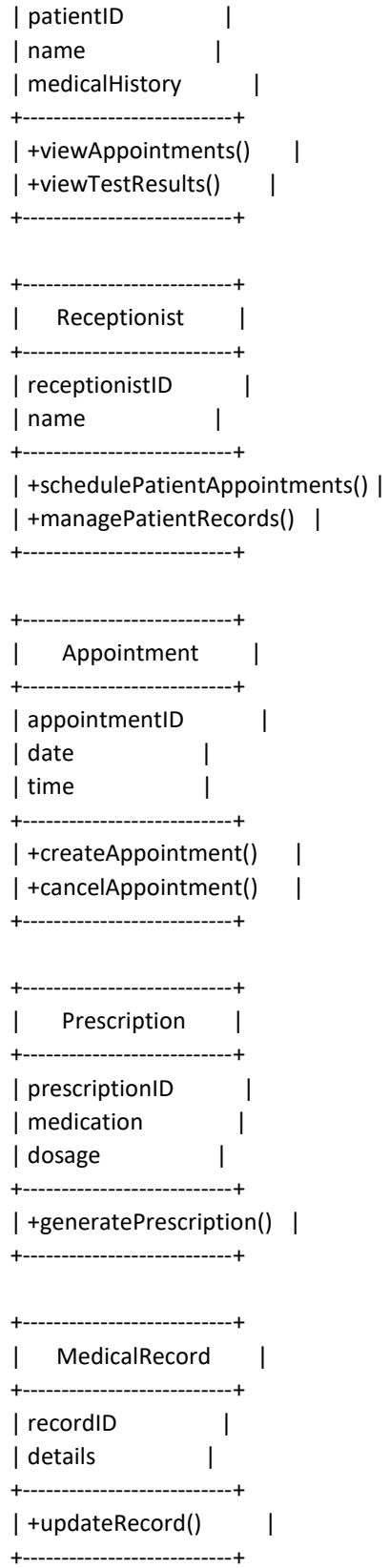
- Attributes: patientID, name, medicalHistory
- Methods: viewAppointments(), viewTestResults()

4. Receptionist
   - Attributes: receptionistID, name
   - Methods: schedulePatientAppointments(), managePatientRecords()
5. Appointment
   - Attributes: appointmentID, date, time
   - Methods: createAppointment(), cancelAppointment()
6. Prescription
   - Attributes: prescriptionID, medication, dosage
   - Methods: generatePrescription()
7. MedicalRecord
   - Attributes: recordID, details
   - Methods: updateRecord()

Relationships:

1. Association between Doctor and Patient
2. Aggregation between Hospital and Department
3. Composition between Patient and MedicalRecord
4. Inheritance between Doctor and Specialist
5. Dependency between Prescription and Doctor, Patient





This class diagram represents the structure of the hospital management system and the relationships between its different entities.

Q. You have been appointed a project manager for a small software products company Your job is to build a product that combines virtual reality hardware with state-of-the art software. Because competition for the home entertainment market is intense, there is significant pressure to get the jols done it short duration with good quality and high user satisfaction. Prescribe success criteria of this project and determine how you will fulfill the success criteria of this product by considering P's of project management.

Success Criteria for the VR Product Project

1. **Product Quality**: The VR product must meet high standards of reliability, functionality, and usability.
2. **User Satisfaction**: The product must be user-friendly, immersive, and meet or exceed customer expectations.
3. **Timely Delivery**: The project must be completed within the set deadline to gain a competitive advantage.
4. **Budget Compliance**: The project should be completed within the allocated budget.
5. **Market Competitiveness**: The product must be competitive in terms of features, performance, and price.
6. **Scalability and Future-Proofing**: The product should be scalable and easily upgradable to adapt to future technological advancements.
7. **Compliance with Standards**: The product must comply with industry standards and regulations.
8. **Stakeholder Satisfaction**: Meeting the expectations of all stakeholders, including customers, investors, and team members.

Fulfilling the Success Criteria Using the P's of Project Management

1. Purpose

- **Define Clear Objectives**: Establish clear, measurable objectives for the project. Ensure that these objectives align with the company's strategic goals and the needs of the home entertainment market.
- **User-Centric Design**: Focus on creating an immersive and satisfying user experience. Conduct market research to understand user needs and preferences.

2. Plan

- **Comprehensive Planning**: Develop a detailed project plan outlining the scope, timeline, resources, and budget. Use project management tools to track progress and adjust plans as needed.
- **Agile Methodology**: Implement agile project management practices to allow for flexibility and iterative development. This helps in adapting to changes quickly and improving the product incrementally.

3. Process

- **Effective Communication**: Establish clear communication channels among team members, stakeholders, and customers. Regular updates and feedback loops are crucial.
- **Quality Assurance**: Implement a robust quality assurance process, including regular testing and validation at every stage of development. Ensure that both hardware and software components meet quality standards.
- **Risk Management**: Identify potential risks early and develop mitigation strategies. Regularly review and update the risk management plan.

4. People

- **Skilled Team**: Assemble a team of skilled professionals with expertise in VR hardware, software development, user experience design, and project management.
- **Training and Development**: Provide continuous training and development opportunities to keep the team updated with the latest technologies and best practices.
- **Motivation and Engagement**: Keep the team motivated and engaged by recognizing achievements, providing a supportive work environment, and fostering collaboration.

5. Performance

- **Performance Metrics**: Define and monitor key performance indicators (KPIs) such as project progress, quality metrics, budget adherence, and user satisfaction.

- **Continuous Improvement**: Encourage a culture of continuous improvement by regularly reviewing performance and implementing feedback.

6. Promotion

- **Marketing Strategy**: Develop a strong marketing strategy to create awareness and excitement about the product. Highlight unique features and benefits that differentiate the product from competitors.
- **Customer Engagement**: Engage with potential customers through demos, trials, and feedback sessions to build anticipation and gather valuable insights for further improvements.

By carefully managing these aspects, you can ensure the successful delivery of a high-quality, user-satisfying VR product within the competitive home entertainment market.

(imp) Q. Explain different testing strategies and testing techniques used in software engineering.

Testing Strategies:

Big Bang Testing

- **Description**: Big Bang Testing is a traditional approach where testing occurs after all or most of the development is completed.
- **Purpose**: It aims to identify defects and validate the entire system as a whole.
- **Advantages**: Simple to implement for small projects; however, it can be challenging to pinpoint the source of defects when issues arise.

Incremental Testing

Incremental testing involves testing progressively as components or modules are integrated into the system. It can be divided into several types:

Unit Testing

- **Description**: Unit Testing focuses on testing individual units or components of the software independently.
- **Purpose**: Verify that each unit works as expected and meets its specifications.
- **Advantages**: Early detection of defects at a granular level, facilitating easier debugging and maintenance.

Integration Testing

Integration Testing verifies the interactions between integrated components or modules. It can be approached in various ways:

- **Top-Down Approach**:
    - **Description**: Testing starts from the topmost module in the hierarchy and progressively integrates lower-level modules.
    - **Purpose**: Identify integration issues early and ensure that the system functions correctly as a whole.
- **Bottom-Up Approach**:
    - **Description**: Testing begins with testing lower-level modules first, gradually integrating higher-level modules.
    - **Purpose**: Validate individual components and ensure they integrate correctly to form larger subsystems.
- **Regression Testing**:
    - **Description**: Ensures that previously developed and tested software still performs correctly after changes or enhancements.
    - **Purpose**: Detect regressions (unexpected behavior introduced by changes) and ensure software stability and reliability.

Validation Testing

Validation Testing evaluates the software against business requirements to ensure it meets user needs. It includes:

- **Alpha Testing**:

    - **Description**: Conducted by the development team within the organization before releasing the software to external users.
    - **Purpose**: Identify usability issues, gather feedback, and ensure the software meets internal quality standards.
- **Beta Testing**:
    - **Description**: Conducted by a select group of external users in a real-world environment.
    - **Purpose**: Gather user feedback, identify issues not caught in earlier testing phases, and validate the software's readiness for release.

System Testing

System Testing evaluates the behavior of the entire system against specified requirements. It includes:

- **Recovery Testing**:
    - **Description**: Tests the system's ability to recover from crashes, hardware failures, or other disruptions.
    - **Purpose**: Ensure data integrity and system availability under adverse conditions.
- **Security Testing**:
    - **Description**: Identifies vulnerabilities and weaknesses in the system's security controls.
    - **Purpose**: Protect against unauthorized access, data breaches, and other security threats.
- **Stress Testing**:
    - **Description**: Tests the system's performance under extreme conditions such as high loads or resource constraints.
    - **Purpose**: Determine the system's robustness, scalability, and reliability under stressful conditions.
- **Performance Testing**:
    - **Description**: Evaluates the system's responsiveness, speed, and stability under various workload conditions.
    - **Purpose**: Identify performance bottlenecks, optimize system performance, and ensure it meets performance requirements.

Testing Techniques:

Testing techniques in software engineering are categorized into two main approaches: black-box testing and white-box testing. These techniques are fundamental to ensuring software quality and are applied at different stages of the software development lifecycle. Let's explore each in detail:

Black-box Testing

- **Description**: Black-box testing, also known as behavioral testing or functional testing, focuses on testing the functionality of a software application without considering its internal code structure, implementation details, or logic.
- **Approach**: Testers interact with the software through its external interfaces (e.g., GUI, APIs) and inputs to validate expected outputs and behaviors.
- **Characteristics**:
    - Tests are based on the software's specifications, requirements, and expected behavior.
    - Test cases are designed without knowledge of the internal workings of the software.
    - Typically used for higher-level testing such as system testing, acceptance testing, and regression testing.
- **Advantages**:
    - Tests are based on user perspectives and requirements.
    - Testers do not need knowledge of programming languages or implementation details.
    - Effective in detecting issues related to functionality, usability, and performance from a user's viewpoint.

- **Disadvantages**:
    - May overlook certain code-level defects or logic errors not directly observable from external interfaces.
    - Limited control over internal code paths, making it challenging to achieve complete coverage.

White-box Testing

- **Description**: White-box testing, also known as structural testing or glass-box testing, examines the internal structure, code, and logic of a software application.
- **Approach**: Test cases are derived from understanding the internal structure of the software, including control flow, data flow, and code paths.
- **Characteristics**:
    - Tests are based on the software's internal design, implementation, and code coverage criteria.
    - Requires knowledge of programming languages and familiarity with the software's architecture.
    - Used for unit testing, integration testing, and code coverage analysis.
- **Advantages**:
    - Provides thorough coverage of code paths, helping to uncover complex logic errors, boundary conditions, and performance bottlenecks.
    - Allows for early detection and resolution of defects during development.
    - Supports metrics such as code coverage to assess testing completeness.
- **Disadvantages**:
    - Test cases may be biased towards known paths and may miss unexpected behaviors or interactions.
    - Testing can be time-consuming and requires detailed understanding of the software's internal workings.
    - Changes in code structure may require extensive test case modifications.

Choosing Between Black-box and White-box Testing

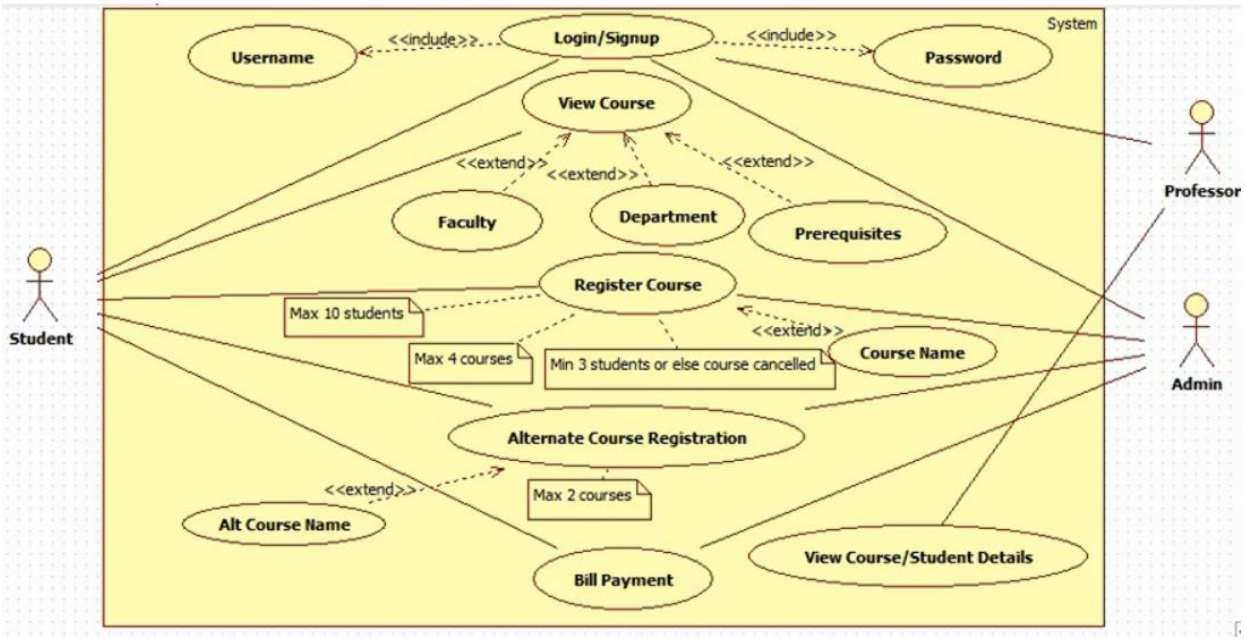
- **Application**:
    - Use black-box testing for validating functional requirements and user scenarios, especially during system and acceptance testing phases.
    - Employ white-box testing for verifying code correctness, uncovering implementation issues, and ensuring comprehensive test coverage during unit and integration testing phases.
- **Complementary Approach**:
    - Often, a combination of both techniques (gray-box testing) is used to leverage their strengths and compensate for their limitations.
    - Gray-box testing combines aspects of black-box and white-box testing to achieve broader test coverage and thorough defect detection.

Conclusion

Black-box and white-box testing are essential techniques in software testing, each offering distinct advantages and challenges. Choosing the appropriate technique depends on the testing objectives, stage of development, and desired coverage. Effective testing strategies often incorporate both approaches to ensure comprehensive validation and verification of software functionality, reliability, and performance before deployment.

Past Papers:

Q. You have been assigned to software team that is developing a network based course registration system. Draw a use case diagram for proposed system.



| Course Registration System |

|

| Actors |

|---|

| Student |

| Administrator |

|

| Use Cases |

|---|

| Browse Courses |

| Search Courses |

| View Course Details |

| Register for Course |

| Drop Course |

| Modify Registration |

| Manage Courses |

| Generate Reports |

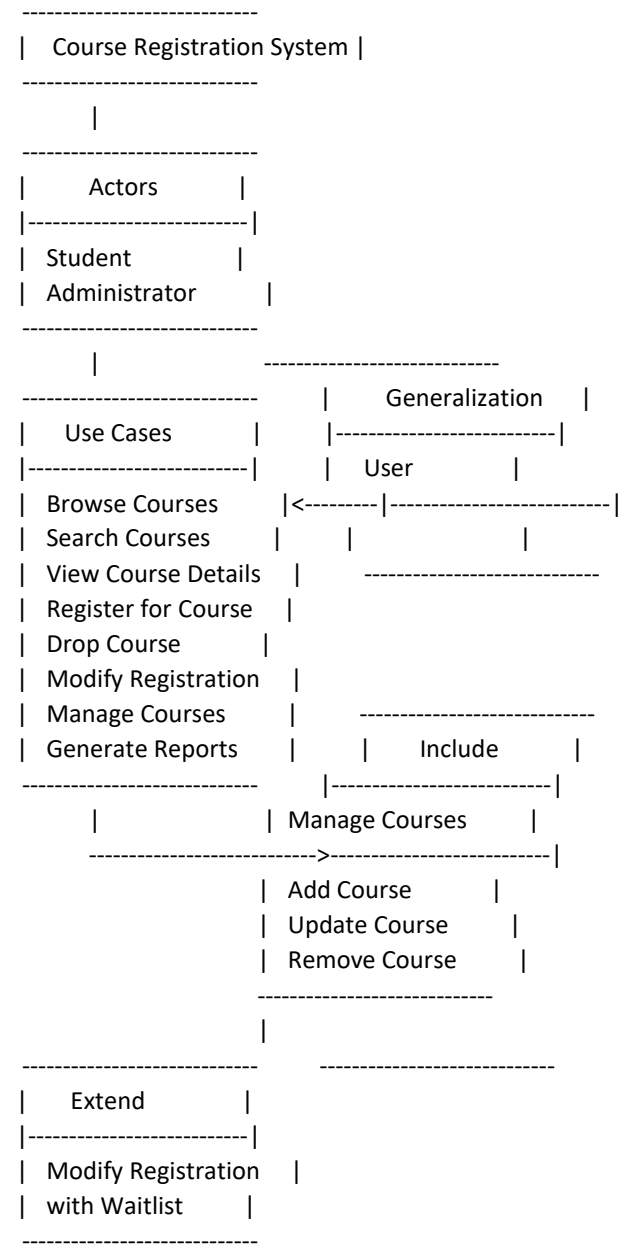
Use Case Descriptions:

1. **Browse Courses**: Allows students to browse available courses.
2. **Search Courses**: Enables students to search for specific courses based on criteria.
3. **View Course Details**: Provides detailed information about a selected course.
4. **Register for Course**: Allows students to register for courses they are interested in.
5. **Drop Course**: Enables students to drop courses they are currently registered in.
6. **Modify Registration**: Allows students to modify their course registrations (e.g., change sections).

7. **Manage Courses**: Allows administrators to manage course offerings, including adding, updating, and removing courses.
8. **Generate Reports**: Enables administrators to generate reports related to course registrations, enrollment statistics, etc.

This diagram outlines the interactions between actors (Student and Administrator) and the system's functionalities (use cases) necessary for a course registration system.

Certainly! Here's an updated use case diagram that includes include, extend relationships, and generalization:
sql
Copy code



Use Case Descriptions:

1. **Browse Courses**: Allows students to browse available courses.
2. **Search Courses**: Enables students to search for specific courses based on criteria.

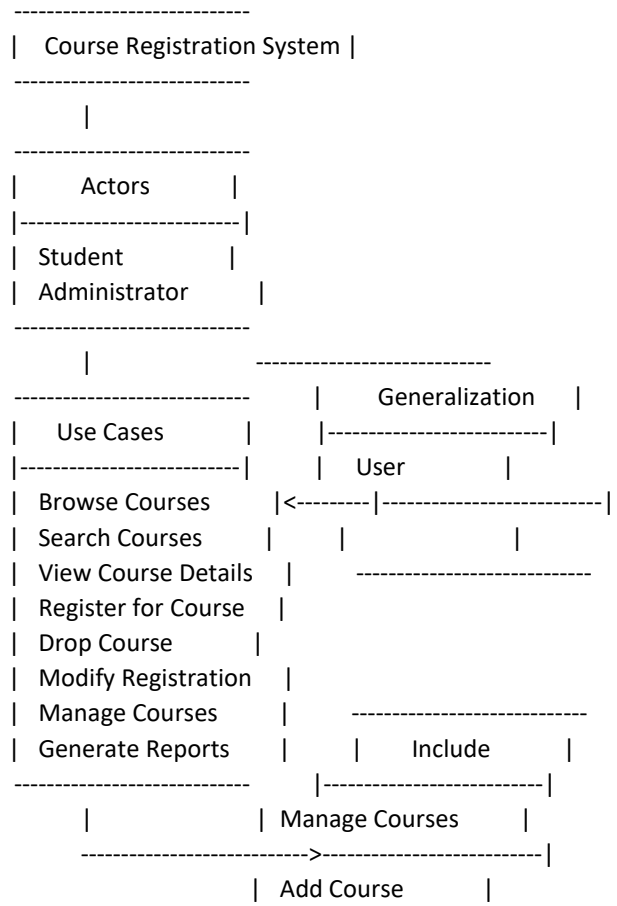
3. **View Course Details**: Provides detailed information about a selected course.
4. **Register for Course**: Allows students to register for courses they are interested in.
5. **Drop Course**: Enables students to drop courses they are currently registered in.
6. **Modify Registration**: Allows students to modify their course registrations. It can be extended with "Waitlist" functionality if the course is full.
7. **Manage Courses**: Allows administrators to manage course offerings. Includes functionalities such as adding, updating, and removing courses.
    - **Add Course**: Specific use case included in "Manage Courses" to add new courses.
    - **Update Course**: Specific use case included in "Manage Courses" to update existing course information.
    - **Remove Course**: Specific use case included in "Manage Courses" to remove courses from the system.
8. **Generate Reports**: Enables administrators to generate reports related to course registrations, enrollment statistics, etc.

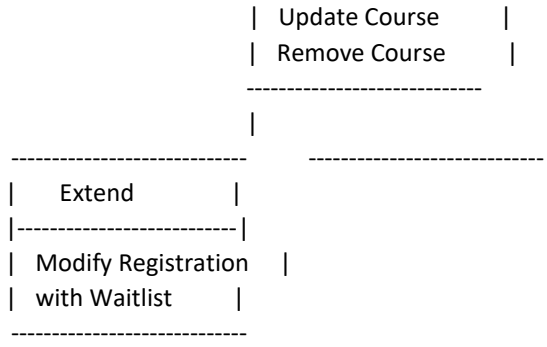
Relationships:

- **Generalization (User)**: Both Student and Administrator actors are generalized as Users of the system.
- **Include (Manage Courses)**: The "Manage Courses" use case includes the specific functionalities of adding, updating, and removing courses.
- **Extend (Modify Registration with Waitlist)**: The "Modify Registration" use case can be extended with "Waitlist" functionality, allowing students to join a waitlist if a course is full.

This diagram now includes relationships such as generalization, include, and extend, providing a more comprehensive view of the interactions between actors and the system's functionalities.

Certainly! Here's an updated use case diagram that includes include, extend relationships, and generalization:





Use Case Descriptions:

1. **Browse Courses**: Allows students to browse available courses.
2. **Search Courses**: Enables students to search for specific courses based on criteria.
3. **View Course Details**: Provides detailed information about a selected course.
4. **Register for Course**: Allows students to register for courses they are interested in.
5. **Drop Course**: Enables students to drop courses they are currently registered in.
6. **Modify Registration**: Allows students to modify their course registrations. It can be extended with "Waitlist" functionality if the course is full.
7. **Manage Courses**: Allows administrators to manage course offerings. Includes functionalities such as adding, updating, and removing courses.
    - **Add Course**: Specific use case included in "Manage Courses" to add new courses.
    - **Update Course**: Specific use case included in "Manage Courses" to update existing course information.
    - **Remove Course**: Specific use case included in "Manage Courses" to remove courses from the system.
8. **Generate Reports**: Enables administrators to generate reports related to course registrations, enrollment statistics, etc.

Relationships:

- **Generalization (User)**: Both Student and Administrator actors are generalized as Users of the system.
- **Include (Manage Courses)**: The "Manage Courses" use case includes the specific functionalities of adding, updating, and removing courses.
- **Extend (Modify Registration with Waitlist)**: The "Modify Registration" use case can be extended with "Waitlist" functionality, allowing students to join a waitlist if a course is full.

This diagram now includes relationships such as generalization, include, and extend, providing a more comprehensive view of the interactions between actors and the system's functionalities.

Q. Give list of practices of extreme programming along with one-line description of each.

Extreme Programming (XP) is a software development methodology that emphasizes frequent releases, customer involvement, and adaptive planning. Here are some key practices of Extreme Programming along with brief descriptions:

1. **Pair Programming**:
    - **Description**: Two programmers work together at one workstation, actively collaborating on the same code.
2. **Test-Driven Development (TDD)**:
    - **Description**: Developers write automated tests before writing the corresponding code, ensuring test coverage and code correctness.
3. **Continuous Integration**:

    - **Description**: Code changes from multiple developers are integrated into a shared repository several times a day, with automated builds and tests.
4. **Refactoring**:
    - **Description**: Restructuring existing code without changing its external behavior, aiming to improve readability, maintainability, and performance.
5. **Simple Design**:
    - **Description**: Keeping designs as simple as possible, avoiding unnecessary complexity and focusing on meeting current requirements.
6. **Collective Code Ownership**:
    - **Description**: Every team member has the responsibility and authority to make changes to any part of the codebase.
7. **Continuous Feedback**:
    - **Description**: Regular communication with stakeholders and users to gather feedback and incorporate it into ongoing development.
8. **Small Releases**:
    - **Description**: Delivering small, incremental updates to the software frequently, ensuring rapid adaptation to changing requirements.
9. **Coding Standards**:
    - **Description**: Establishing and adhering to agreed-upon coding conventions and guidelines to promote consistency and maintainability.
10. **Sustainable Pace**:
    - **Description**: Ensuring that team members work at a comfortable and sustainable pace, avoiding burnout and promoting long-term productivity.
11. **System Metaphor**:
    - **Description**: System metaphor provides a common vision or analogy that helps stakeholders, including developers and customers, understand the system's architecture, design, and functionality.
    - **Purpose**: It serves as a unifying concept that guides development decisions, promotes consistent understanding across the team, and aligns technical choices with business goals.
    - **Example**: Using a "library" metaphor for a content management system where content items are books, users are librarians, and operations are borrowing and returning books.
12. **Whole Team**:
    - **Description**: The whole team approach involves all stakeholders—developers, testers, designers, product owners, and customers—working together collaboratively throughout the development process.
    - **Purpose**: It ensures shared ownership of the project's success, fosters communication and collaboration, and leverages diverse perspectives to deliver high-quality software.
    - **Example**: Regular team meetings involving all stakeholders to discuss progress, challenges, and priorities, ensuring everyone has a voice in decision-making.
13. **Design Improvement**:
    - **Description**: XP encourages continuous design improvement throughout the development lifecycle, prioritizing simplicity, flexibility, and adaptability.
    - **Purpose**: It facilitates evolving requirements and enhances the system's maintainability by iteratively refining the design based on feedback, changing needs, and emerging insights.
    - **Example**: Conducting regular design reviews or refactorings to address evolving requirements or to optimize performance and scalability.
14. **Planning Game**

Description: The Planning Game is a collaborative process where developers and customers work together to plan and prioritize development tasks. It typically involves two main activities: Release Planning and Iteration Planning.
Purpose: The Planning Game ensures that development efforts are aligned with business priorities, promotes transparency and collaboration between the development team and stakeholders, and facilitates adaptive planning based on feedback and changing requirements.

These practices are designed to promote agility, flexibility, and high-quality software development, making Extreme Programming suitable for projects where requirements are likely to change or evolve over time.

Q. Differentiate:

1. **User Requirements vs. System Requirements**:
    - **User Requirements**:
        - These are statements in natural language plus diagrams that describe the services the system should provide to its users.
        - They focus on what the user needs or desires from the system.
        - User requirements are typically written for a non-technical audience and serve as the basis for system design.
    - **System Requirements**:
        - These are detailed descriptions of the system's functions, services, and operational constraints.
        - They describe how the system will fulfill the user requirements.
        - System requirements are typically more technical and specific, detailing the software and hardware components, performance criteria, interfaces, and other technical aspects.
2. **Reactive Risk Strategy vs. Proactive Risk Strategy**:
    - **Reactive Risk Strategy**:
        - This strategy involves responding to risks after they have occurred.
        - It focuses on managing and mitigating risks once they become apparent and threaten project objectives.
        - Examples include contingency plans, issue resolution, and crisis management.
    - **Proactive Risk Strategy**:
        - This strategy involves identifying and addressing potential risks before they become issues.
        - It focuses on risk prevention and early mitigation through risk identification, analysis, and planning.
        - Examples include risk assessments, risk avoidance techniques, and risk management processes integrated into project planning and execution.
3. **Generic Software Product vs. Customized Software Product**:
    - **Generic Software Product**:
        - These are software products developed to be used by a wide range of users or organizations without modification.
        - They typically have standard features and functionalities that cater to common needs.
        - Examples include off-the-shelf software like Microsoft Office or Adobe Photoshop.
    - **Customized Software Product**:
        - These are software products tailored or modified to meet specific requirements of a particular user or organization.
        - They involve customization of features, functionalities, and sometimes the user interface to align closely with the user's unique needs.

        - Examples include enterprise resource planning (ERP) systems customized for specific industries or organizations.

These differentiations help clarify the distinctions between each pair of terms, highlighting their respective focuses and applications in software development and project management.

(imp) Q. Explain all design concepts wit short description for each.

Design concepts in software engineering encompass fundamental principles and guidelines that guide the creation of efficient, maintainable, and scalable software systems. Here's a brief explanation of each design concept:

1. **Abstraction**:
    - **Description**: Abstraction focuses on hiding complex implementation details and exposing only relevant functionalities to the user. It helps manage complexity by providing a simplified view of entities and operations.
    - **Data Abstraction**: Data abstraction focuses on hiding implementation details of data objects and exposing only essential features, such as operations or attributes, to manipulate them. It enhances security, simplifies programming, and improves maintainability.
    - **Procedural Abstraction**: Procedural abstraction hides the implementation details of procedures (functions or methods), providing a clear interface for invoking operations without needing to understand their internal workings.
2. **Architecture**:
    - **Description**: Architecture in software engineering refers to the fundamental structure of a software system, including its components, relationships, and principles guiding its design and evolution. It defines the overall organization to ensure functionality, reliability, scalability, and maintainability.
3. **Design Pattern**:
    - **Description**: A design pattern is a reusable solution to a commonly occurring problem in software design. Patterns provide proven approaches to design and structure code, promoting code reusability, maintainability, and scalability. Examples include Singleton, Factory Method, Observer, and MVC.
4. **Separation of Concerns**:
    - **Description**: Separation of concerns (SoC) involves dividing a software system into distinct sections (concerns), each addressing a separate aspect of functionality. It enhances modularity, maintainability, and readability by reducing complexity and allowing focused development and testing.
5. **Modularity**:
    - **Description**: Modularity refers to the practice of dividing software into separate, interchangeable components (modules), each responsible for a specific set of functionalities. It promotes reusability, ease of maintenance, and scalability by encapsulating related features and minimizing dependencies.
6. **Information Hiding**:
    - **Description**: Information hiding restricts access to internal details of a software module, exposing only necessary interfaces or abstractions. It enhances security, prevents unintended dependencies, and facilitates changes to internal implementation without affecting external functionality.
7. **Refinement**:
    - **Description**: Refinement involves progressively elaborating the design and implementation of software components from abstract concepts to detailed specifications. It ensures that high-level requirements are translated into concrete, executable solutions, aligning with user needs and system constraints.
8. **Aspect**:
    - **Description**: In aspect-oriented programming (AOP), an aspect is a modular unit of cross-cutting concerns, such as logging, security, or transaction management. AOP separates concerns that

would otherwise be tangled or scattered throughout the codebase, improving modularity and maintainability.

9. **Refactoring**:
    - **Description**: Refactoring is the process of restructuring existing code without changing its external behavior. It improves code readability, maintainability, and performance by eliminating code smells (such as duplicated code or complex logic) and applying design patterns.
10. **Functional Independence (Cohesion and Coupling)**:
    - **Cohesion**: Cohesion measures how closely related elements within a module are. High cohesion means elements within a module perform a single, well-defined task. It enhances module clarity and reduces complexity.
    - **Coupling**: Coupling measures the degree of interdependence between modules. Loose coupling reduces dependencies between modules, making the system more flexible, reusable, and easier to maintain.

These concepts collectively contribute to the design, development, and maintenance of high-quality software systems, ensuring they meet functional requirements while adhering to best practices in software engineering.

Q. Suppose you are involved in a large project concerning the development of a patient planning system for a hospital. You may opt for one of the two strategies. The first strategy is to start with a thorough analysis of user requirements, after which the system is built according to these requirements. The second strategy starts with a less complete requirements analysis phase, after which a pilot version is developed. This pilot version is installed in a few small departments. Further developments of the system are guided by the experience gained in working with the pilot version. Discuss the pros and cons of both strategies. Which strategy do you favor?

Strategy 1
Requirements gathering can be exausting, they may take way longer than anyone expected and delay the commencement of the development process. The requirements may also keep changing frequently. The customer wants a tech solution now and not 2 years later.
Pro:
You get the user to clearly agree on certain terms and conditions. (Revisions, features)
Cons:
Users may not know what exactly they want.

Strategy 2
More often that not, the customer will be unsure as to what they exactly need until you show it to them. Nobody needed an ipad before it came into existence, right? So my pick would definitely be the second strategy. Obviously, there are chances of you not even falling in the same ballpark with your pilot version. But you'll spend more time doing than documenting and finalising.
Pro:
Pilot will serve as a POC rather than documentation which will be more appealing.
Cons:
Lots of revisions

Q. Discuss different perspective of analysis modeling.

Analysis modeling in software engineering encompasses various perspectives that help in understanding and defining system requirements and behavior. Here are different perspectives of analysis modeling:

1. **Structural Perspective**:
    - **Class Diagrams**: Represent the static structure of the system, showing classes, attributes, relationships, and methods.
    - **Object Diagrams**: Provide a snapshot of instances of classes and their relationships at a specific point in time.

    - **Entity-Relationship Diagrams (ERD)**: Depict entities, their attributes, and relationships in a database context.
    - **Data Flow Diagrams (DFD)**: Illustrate how data flows through a system, showing processes, data stores, and data flows.
2. **Behavioral Perspective**:
    - **Use Case Diagrams**: Define the functional requirements of the system from the user's perspective, depicting actors, use cases, and their relationships.
    - **Sequence Diagrams**: Show interactions between objects over time, illustrating the sequence of messages exchanged.
    - **State Machine Diagrams**: Describe the behavior of an object in terms of states, transitions between states, and events triggering transitions.
    - **Activity Diagrams**: Represent workflows or processes, showing activities, decision points, and transitions between activities.
3. **Interaction Perspective**:
    - **Sequence Diagrams**: Highlight interactions between objects in a chronological sequence.
    - **Communication Diagrams**: Show how objects collaborate to achieve specific functionalities, emphasizing the structural organization of objects and their interactions.

Each perspective offers a unique view into the system under analysis, allowing stakeholders to comprehend different aspects such as structure, behavior, functionality, and control flow. These models serve as blueprints for design and implementation, guiding software engineers throughout the development lifecycle to ensure that the final product meets the desired requirements and expectations.

Structural Perspective

Class Diagrams:

- **Description**: Represent the static structure of the system, depicting classes, attributes, relationships, and methods.
- **Purpose**: Provide an overview of the system's structure, helping to understand the organization and relationships between classes.

Object Diagrams:

- **Description**: Provide a snapshot of instances of classes and their relationships at a specific point in time.
- **Purpose**: Illustrate how objects interact and collaborate during runtime, aiding in understanding system behavior.

Entity-Relationship Diagrams (ERD):

- **Description**: Depict entities, their attributes, and relationships in a database context.
- **Purpose**: Model data entities and their relationships to design and optimize database structures.

Data Flow Diagrams (DFD):

- **Description**: Illustrate how data flows through a system, showing processes, data stores, and data flows.
- **Purpose**: Clarify how information moves through the system, highlighting inputs, processes, and outputs.

Behavioral Perspective

Use Case Diagrams:

- **Description**: Define functional requirements from the user's perspective, showing actors, use cases, and their relationships.
- **Purpose**: Communicate system functionality and user interactions, guiding requirements gathering and validation.

Sequence Diagrams:

- **Description**: Show interactions between objects over time, illustrating the sequence of messages exchanged.
- **Purpose**: Detail how objects collaborate to achieve specific tasks or scenarios, focusing on dynamic behavior.

State Machine Diagrams:

- **Description**: Describe the behavior of an object in terms of states, transitions between states, and events triggering transitions.
- **Purpose**: Model complex behaviors and state changes within objects or systems, aiding in understanding system logic.

Activity Diagrams:

- **Description**: Represent workflows or processes, showing activities, decision points, and transitions between activities.
- **Purpose**: Visualize the flow of activities and decision points in a process or workflow, facilitating process analysis and optimization.

Interaction Perspective

Sequence Diagrams (Interaction Perspective):

- **Description**: Highlight interactions between objects in a chronological sequence.
- **Purpose**: Detail the sequence of messages exchanged between objects or components, clarifying the timing and flow of interactions.

Communication Diagrams:

- **Description**: Show how objects collaborate to achieve specific functionalities, emphasizing the structural organization of objects and their interactions.
- **Purpose**: Visualize the relationships and interactions between objects, focusing on how they communicate and work together to achieve system goals.

External Perspective

External Perspective:

- **Description**: Focus on the system's interactions with external entities such as users, other systems, or devices.
- **Purpose**: Highlight external interfaces, inputs, outputs, and interactions, ensuring alignment with external requirements and stakeholders' expectations.

Each perspective offers a distinct viewpoint into the system being analyzed, providing stakeholders with valuable insights into different aspects of the software's structure, behavior, interactions, and external interfaces. These models collectively guide software engineers throughout the development lifecycle, ensuring that the final product meets functional requirements and user expectations.

Q. Explain the use of class diagram, describe the types of relationships used in class diagram, define the concept of cardinality and verify all your given concepts through suitable diagram.

Class Diagrams

Class diagrams are fundamental to object-oriented analysis and design, illustrating the static structure of a system by depicting classes, their attributes, operations (methods), and relationships. They provide a blueprint for the system's architecture and are widely used in both analysis and design phases of software development.

Types of Relationships in Class Diagrams

1. **Association**:
    - **Description**: Represents a structural relationship between classes. It indicates that objects of one class are connected to objects of another class.
    - **Example**: A Customer class may be associated with an Order class, indicating that customers place orders.
2. **Aggregation**:
    - **Description**: Represents a "whole-part" relationship where a part (child) class is part of a whole (parent) class. The part can exist independently of the whole.

    - **Example**: A Department class aggregates Employee classes, meaning employees belong to a department but can exist independently.
3. **Composition**:
    - **Description**: Represents a stronger form of aggregation where the part (child) class cannot exist without the whole (parent) class.
    - **Example**: A Car class composed of Engine, Wheel, and Body classes. The parts (engine, wheels, body) are crucial components of the car.
4. **Generalization (Inheritance)**:
    - **Description**: Represents an "is-a" relationship between a superclass (parent) and a subclass (child). The subclass inherits attributes and behaviors from the superclass.
    - **Example**: A Vehicle class may be a superclass, and Car, Truck, and Motorcycle classes may be subclasses inheriting from Vehicle.

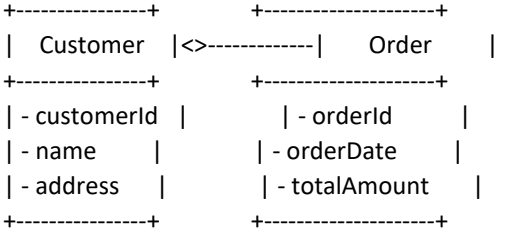
Cardinality in Class Diagrams

Cardinality defines the number of instances of one class (source) that can be associated with the number of instances of another class (target). It specifies the multiplicity or relationship between classes in terms of minimum and maximum occurrences.

- **Multiplicity**: Indicates the exact number of instances related in a specific association.
- **Role**: Specifies the name of the association role for each class.
- **Navigability**: Indicates whether objects of one class can navigate to objects of another class.

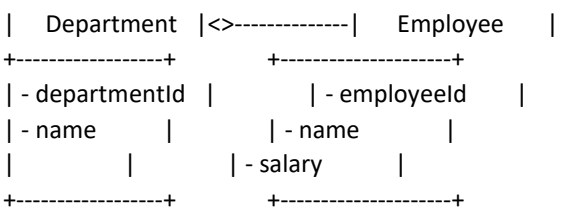
Example Class Diagram

Let's verify these concepts using an example class diagram:



- **Association**: Customer places Order (Customer <>-- Order)
- **Aggregation**: Department aggregates Employee (Department <>-- Employee)
- **Composition**: Car composed of Engine, Wheel, Body (Car <>-- Engine, Wheel, Body)
- **Generalization**: Vehicle is a superclass, Car, Truck, Motorcycle are subclasses (Vehicle <|-- Car, Truck, Motorcycle)

Cardinality Example



- **Multiplicity**: Department has 0 to many Employees (Department <>-- Employee 0..*)
- **Role**: Department and Employee roles are explicitly stated in their respective classes.
- **Navigability**: Indicates the direction of access and navigation between related classes.

Class diagrams are essential tools for visualizing and communicating the structure and relationships within a software system, facilitating effective analysis, design, and implementation.

Past Papers

You have been appointed a project manager within an information systems organization. Your job is to build an application that is quite similar to others your team has built, although this is larger and more complex. Requirements have been thoroughly documented by the customer. What software process model(s) would you choose and why? Explain in detail.

As the information system already exists in which some development is to be made and released. Now Developer just need to make some addition in the previous application because Customer requirements are documented, which mean requirements are well understood and have no need to modified requirement.

Process Selected:
Prototype model will be used in development of this application. In prototype model application or project is made in series of increment throughout project. In prototype model we first make a prototype and give to the customer and make changes in it accordingly as required by the customer. Further increment made after previous ones. Large systems are particularly suitable for prototype model.
Prototype model is a strategy that allows system to be developed in pieces. Prototype model allows the additions in process as per requirements, process change can be implemented.

To make this application we have to use the already existing application and make increments in it and made new more complex and large application Requirements are fulfilled already for this application.

Q. Discuss how structural partitioning can help to make software more maintainable?

Structural partitioning is a technique used in software engineering to divide a system into smaller, manageable components or modules based on their functionality or responsibilities. This approach aims to improve maintainability by organizing code and design in a structured and cohesive manner.

1. **Encapsulation and Information Hiding**:
    - **Encapsulation**: Modules encapsulate functionality and data, hiding implementation details and providing clear interfaces. This reduces complexity for developers outside the module.
    - **Information Hiding**: Exposing only necessary interfaces minimizes dependencies and unintended side effects during changes, allowing developers to focus on specific modules.
2. **Modularity and Reusability**:
    - **Modularity**: Breaking systems into modules with defined responsibilities promotes easier understanding, testing, and modification.
    - **Reusability**: Well-defined modules can be reused, reducing development time and enhancing software reliability through tested components.
3. **Ease of Maintenance and Debugging**:
    - Changes are localized to specific modules, minimizing impact on other parts of the system and simplifying debugging efforts.
    - Clear module boundaries facilitate efficient troubleshooting and maintenance workflows.
4. **Scalability and Extensibility**:
    - **Scalability**: Adding new features is facilitated by extending or introducing new modules, supporting system growth.
    - **Extensibility**: Modular designs enable modifications without disrupting the entire system, crucial for adapting to evolving requirements.
5. **Team Collaboration and Workflow**:
    - **Team Collaboration**: Well-defined modules enable concurrent development by teams, leveraging specialized expertise.

- **Maintenance Workflow**: Maintenance tasks are managed effectively based on module dependencies and criticality, enhancing productivity.

In conclusion, structural partitioning fosters software systems that are easier to understand, maintain, and evolve over time. By organizing functionality into cohesive modules with clear interfaces, software engineers can build robust and adaptable systems that meet both current and future needs effectively.

Q. Write the advantages and disadvantages of waterfall and prototyping model.

Waterfall Model: Advantages and Disadvantages

Advantages:

1. **Clear and Well-Defined Phases**: Each phase has specific deliverables and review processes, making it easy to understand and manage.
2. **Structured Approach**: Progress is easy to track, and milestones are well-defined, which helps in project management and scheduling.
3. **Documentation**: Extensive documentation is produced at each stage, aiding in maintenance and future enhancements.
4. **Suitable for Stable Requirements**: Works well when requirements are well-understood and unlikely to change significantly.

Disadvantages:

1. **Rigid and Sequential**: Little room for iteration or flexibility, making it difficult to accommodate changes late in the process.
2. **Late Feedback**: Stakeholders and users see the product only after completion, increasing the risk of misunderstandings or mismatches with actual needs.
3. **High Risk**: Integration and testing occur late in the process, potentially leading to major issues being discovered late in development.

Prototyping Model: Advantages and Disadvantages

Advantages:

1. **Early Feedback**: Users and stakeholders can interact with prototypes early in the development process, providing valuable feedback.
2. **Flexibility**: Allows for iterative refinement and modification based on feedback and changing requirements.
3. **Risk Reduction**: Issues are identified and resolved early, reducing the risk of costly changes later in development.
4. **Improved Communication**: Enhances communication between development teams and stakeholders by visualizing concepts early.

Disadvantages:

1. **Time and Cost**: Developing prototypes can be time-consuming and may incur additional costs, especially if multiple iterations are needed.
2. **Lack of Documentation**: Prototypes may lack comprehensive documentation, potentially leading to misunderstandings or issues in later stages.
3. **Scope Creep**: Continuous refinement may lead to scope creep if not managed effectively, affecting project timelines and budgets.

Conclusion

Choosing between the waterfall model and prototyping model depends largely on project requirements, timelines, and the level of flexibility needed. Waterfall is suitable for projects with stable and well-understood requirements, while prototyping is beneficial for projects where early feedback and flexibility are crucial. Each model has its strengths and weaknesses, and the choice should be based on the specific characteristics and goals of the project at hand.

Q. Assume that you are the project manager for a company that builds software for consumer products. You have been contracted to build the software for home security system. Write a statement of scope that describes the software.

The security systems installed in the house supports the Home security system. Protects the home from burglars through the use of burglar alarm. It controls the locks by remote control and also detects fire with the help of advanced smoke detectors.
The important aspect of home security is Simulating the presence of the homeowners. In addition, A rich variety of sensors to coordinate, monitor and control as well as emergency notification features is a home security and safety system. The primary user, a homeowner, can define the sequence of lighting, radios, CD players, televisions, security and safety events. For satisfying the homeowner This system will provide these functions.
By using existing alarm monitoring networks this system is designed to be installed by alarm companies and licensed contractors and monitored.

The software for the lawn-mowing robot aims to provide homeowners with an autonomous, efficient, and user-friendly solution for lawn maintenance.
As the project manager for a company building software for household robots, the statement of scope for the software that will be developed for a robot that mows the lawn for a homeowner can be outlined as follows:
Objective: The objective of the software is to enable the robot to autonomously mow the lawn, providing a convenient and time-saving solution for homeowners.
Lawn Navigation: The software will include algorithms and sensors to allow the robot to navigate the lawn efficiently, avoiding obstacles such as trees, flower beds, and furniture.
Cutting Patterns: The software will determine optimal cutting patterns for the lawn, ensuring even and consistent coverage. This may include options for different patterns, such as straight lines or spirals.
Boundary Detection: The robot will be equipped with sensors to detect the boundaries of the lawn, ensuring that it stays within the designated area and does not venture into neighboring properties or other restricted areas.

Q. Explain the input and outputs of Domain analysis.

Domain analysis is a crucial step in the software development process that involves understanding the problem domain to create a more effective and reusable software solution. It focuses on identifying commonalities and variabilities within a particular domain, which can then be used to design a system that is both robust and adaptable. Here's a detailed explanation of the inputs and outputs of domain analysis:

Inputs of Domain Analysis

1. **Domain Knowledge**
    - **Domain Experts**: Information from individuals who have extensive knowledge and experience in the specific domain.
    - **Existing Documentation**: Manuals, guidelines, standards, and existing system documentation related to the domain.
2. **Stakeholder Requirements**
    - **Interviews and Surveys**: Collecting requirements through direct interaction with stakeholders.
    - **Questionnaires**: Structured forms to gather detailed requirements and expectations.
3. **Existing Systems and Software**
    - **Legacy Systems**: Analysis of current systems in use, understanding their functionalities, strengths, and weaknesses.
    - **Competitive Products**: Studying similar products in the market to identify common features and potential improvements.
4. **Regulatory and Compliance Information**

- **Standards and Regulations**: Understanding legal and regulatory requirements that the system must adhere to.

Outputs of Domain Analysis

1. **Domain Model**
   - **Conceptual Models**: Diagrams and descriptions that represent the key concepts, entities, and relationships within the domain.
   - **Taxonomies and Ontologies**: Classification schemes that define the hierarchy and relationships between different domain concepts.
2. **Requirements Specifications**
   - **Functional Requirements**: Detailed descriptions of the functions the system must perform.
   - **Non-functional Requirements**: Performance, usability, reliability, and other quality attributes.
3. **Reusable Components and Patterns**
   - **Common Modules**: Identified software components that can be reused across different projects within the domain.
   - **Design Patterns**: Best practices and patterns that can be applied to solve common problems within the domain.
4. **Glossary of Terms**
   - **Terminology Definitions**: A comprehensive list of domain-specific terms and their meanings to ensure clear communication among stakeholders and developers.
5. **Domain Constraints and Rules**
   - **Business Rules**: Specific rules and constraints that govern the behavior of the system within the domain.
   - **Policies and Procedures**: Established policies and procedures that the system needs to follow.

Example Scenario: E-commerce Domain Analysis

Inputs:

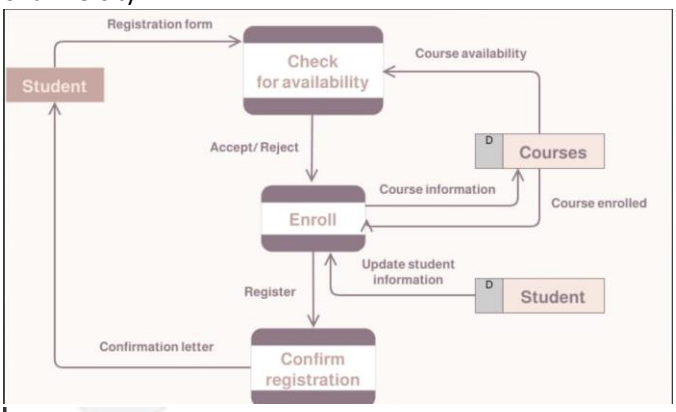
- **Domain Knowledge**: Information from retail experts, existing e-commerce systems documentation.
- **Stakeholder Requirements**: Feedback from customers, business owners, and regulators.
- **Existing Systems**: Analysis of leading e-commerce platforms like Amazon, eBay.
- **Regulatory Information**: Compliance with GDPR, payment processing regulations.

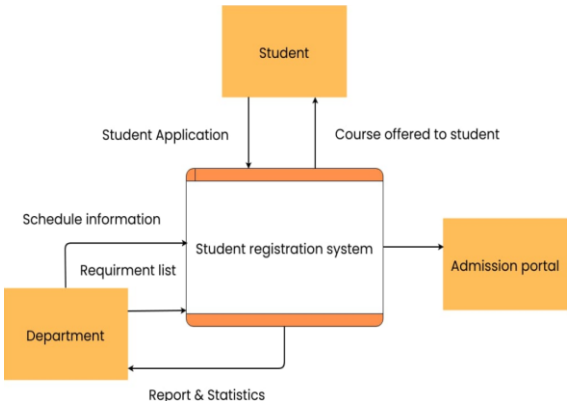
Outputs:

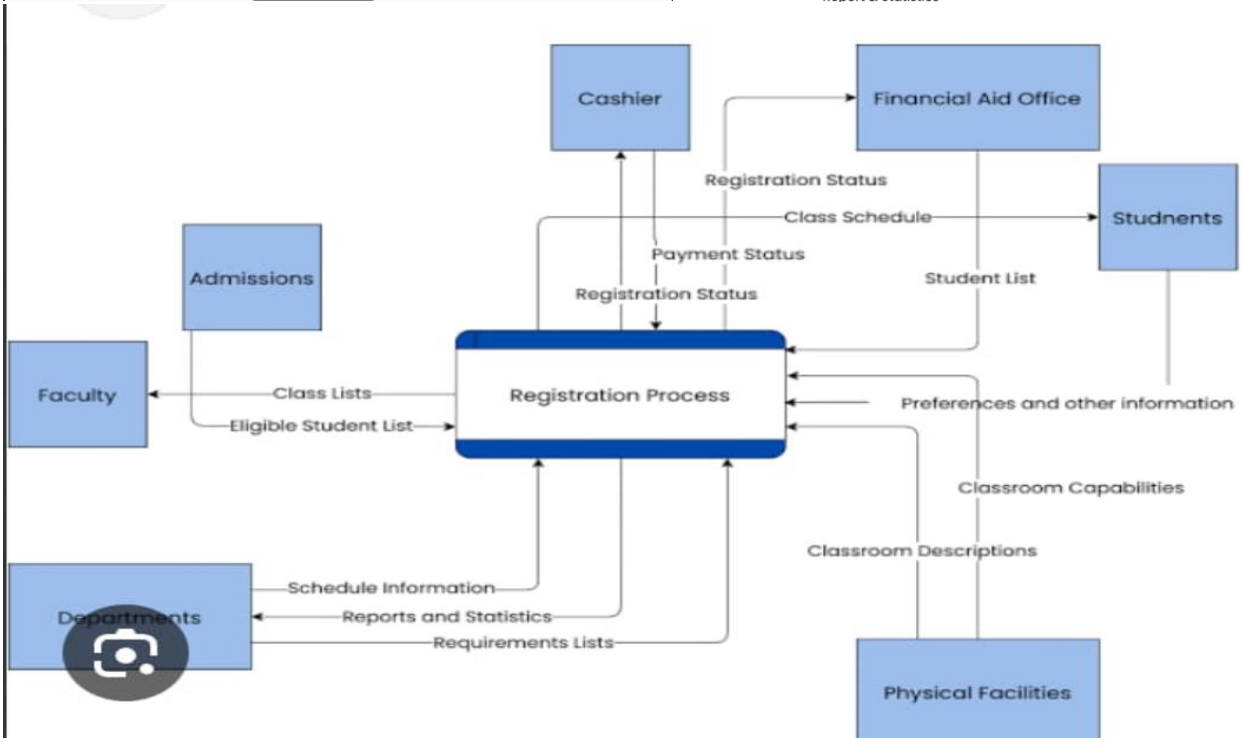
- **Domain Model**:
  - **Conceptual Model**: Diagram representing entities like Product, Customer, Order, and their relationships.
  - **Taxonomy**: Classification of products into categories and subcategories.
- **Requirements Specifications**:
  - **Functional**: Features like product browsing, cart management, checkout process.
  - **Non-functional**: Security, performance metrics for page load times.
- **Reusable Components**:
  - **Common Modules**: Payment processing module, user authentication.
  - **Design Patterns**: Singleton pattern for managing the shopping cart instance.
- **Glossary of Terms**: Definitions for terms like SKU, checkout, and order fulfillment.
- **Domain Constraints and Rules**:
  - **Business Rules**: Discount rules, return policies.
  - **Policies**: Data retention policies for customer information.

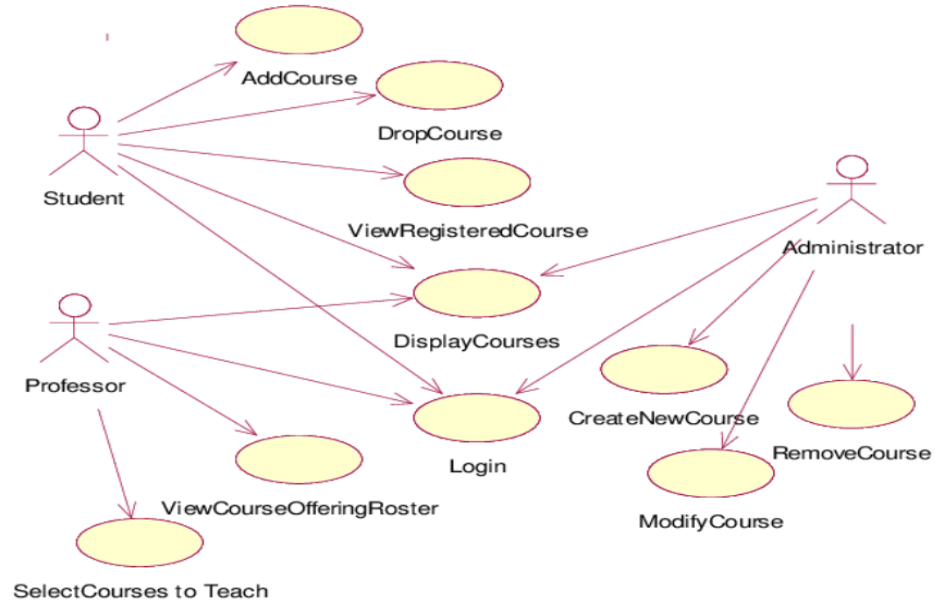
In summary, domain analysis involves gathering and analyzing a wide range of inputs to produce a detailed understanding of the domain, which can then be used to create a robust and flexible software system tailored to that domain's specific needs and constraints.

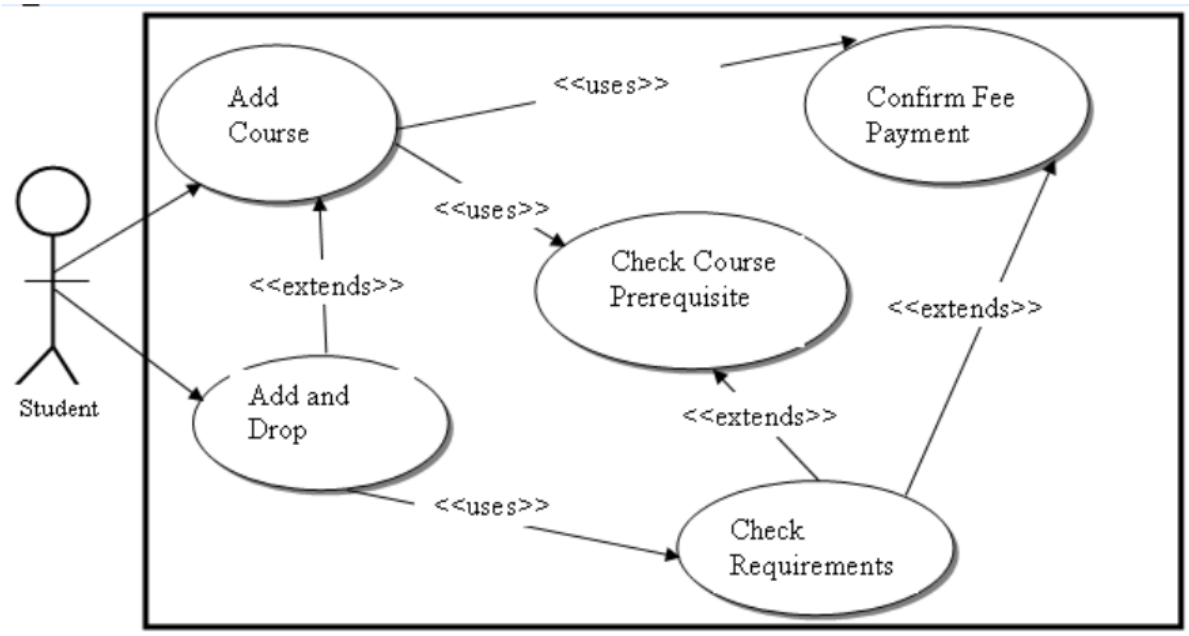
Q. Develop a use case diagram. Use case text and data flow diagram for network based course registration system of university.











Q. Discuss software engineering as a layered technology.

Software engineering is a fully layered technology, to develop software we need to go from one layer to another. All the layers are connected and each layer demands the fulfillment of the previous layer. Fig: The diagram shows the layers of software

Tools

Method

Process

A quality focus

development.

Q. Propose a specific software project that would be amenable to the spiral model, present a scenario for applying the model to the software.

The Spiral Model is particularly well-suited for complex, large-scale software projects that are subject to frequent changes and where risk management is a critical factor. One such project could be the development of a **Virtual Reality (VR) Home Entertainment System**.

Scenario: VR Home Entertainment System

Project Description:

Develop a VR Home Entertainment System that provides users with an immersive experience for movies, games, and virtual social interactions. The system should integrate with existing VR hardware, offer a content library, support multiplayer interactions, and ensure a high level of security and performance.

Applying the Spiral Model:

The Spiral Model involves iterative development through repeated cycles (or spirals), with each cycle encompassing the following phases: Planning, Risk Analysis, Engineering, and Evaluation.

1. First Spiral: Concept Development

- **Planning:**
  - Define the project goals and constraints.
  - Identify key stakeholders (e.g., users, VR hardware manufacturers, content providers).
  - Gather high-level requirements through initial meetings and brainstorming sessions.
- **Risk Analysis:**
  - Identify potential risks such as technical feasibility, market acceptance, and integration challenges with VR hardware.
  - Develop mitigation strategies for each identified risk (e.g., feasibility studies, market research).
- **Engineering:**
  - Create a conceptual prototype of the VR interface.
  - Develop initial use case diagrams and a high-level architecture.
- **Evaluation:**
  - Present the prototype to stakeholders for feedback.
  - Conduct initial user testing with a small group of potential users.
  - Refine requirements based on feedback and identify new risks.

2. Second Spiral: Architectural Design

- **Planning:**
  - Refine project requirements based on feedback from the first spiral.
  - Develop detailed project plans and schedules.
- **Risk Analysis:**
  - Assess risks associated with architectural decisions, such as scalability and compatibility with various VR hardware.
  - Plan for potential technological changes and future scalability.
- **Engineering:**
  - Design the system architecture, including key components such as the VR content library, user interface, and multiplayer support.
  - Develop detailed class diagrams and sequence diagrams.
- **Evaluation:**
  - Review the architectural design with stakeholders and conduct peer reviews.
  - Create architectural prototypes to test critical components.
  - Adjust the architecture based on evaluation results and identify additional risks.

3. Third Spiral: Detailed Design and Initial Development

- **Planning:**

    - Develop detailed designs for each component of the system.
    - Create a detailed project plan for the initial development phase.
- **Risk Analysis:**
    - Identify risks related to detailed design and implementation, such as integration issues and performance bottlenecks.
    - Plan for extensive testing and debugging.
- **Engineering:**
    - Implement the core features of the system, including VR content library, user interface, and basic multiplayer functionality.
    - Develop unit tests and integration tests.
- **Evaluation:**
    - Conduct thorough testing of the initial implementation.
    - Perform usability testing with a broader user group.
    - Collect feedback and refine the design based on testing results.

4. Fourth Spiral: Full Development and Integration

- **Planning:**
    - Update the project plan to reflect insights and adjustments from previous spirals.
    - Plan for full-scale development and integration of advanced features.
- **Risk Analysis:**
    - Assess risks related to full-scale development, such as data security and user privacy.
    - Develop strategies to mitigate these risks, including encryption and secure authentication.
- **Engineering:**
    - Complete the development of all system features, including advanced VR interactions, social features, and content management.
    - Integrate all components and conduct system-level testing.
- **Evaluation:**
    - Perform extensive system testing, including performance testing and security testing.
    - Conduct beta testing with a large group of users.
    - Gather feedback and make necessary adjustments.

5. Fifth Spiral: Deployment and Maintenance

- **Planning:**
    - Develop a deployment plan, including user training and support plans.
    - Plan for ongoing maintenance and updates.
- **Risk Analysis:**
    - Identify risks related to deployment, such as user adoption and post-deployment issues.
    - Plan for a phased deployment to manage potential issues.
- **Engineering:**
    - Deploy the system to users.
    - Provide training and support to ensure smooth adoption.
- **Evaluation:**
    - Monitor system performance and user feedback.
    - Address any issues that arise and plan for future enhancements.

Benefits of Using the Spiral Model for This Project:

1. **Risk Management:** The iterative nature of the Spiral Model allows for continuous risk assessment and mitigation, which is crucial for a complex project with many uncertainties.
2. **Flexibility:** The model accommodates changes in requirements and design, which is important for a project involving cutting-edge technology like VR.

3. **Stakeholder Involvement:** Frequent evaluations and feedback loops ensure that stakeholder needs and expectations are continually addressed.
4. **Incremental Development:** The project progresses through incremental builds, allowing for early detection of issues and continuous improvement.

By applying the Spiral Model to the development of a VR Home Entertainment System, the project can effectively manage risks, incorporate stakeholder feedback, and adapt to changing requirements, ultimately leading to a more successful and robust product.

Q. List the Lean software development principles.

Lean software development is an adaptation of lean manufacturing principles and practices to the software development domain. It aims to increase efficiency and reduce waste. Here are the key principles of Lean software development:

1. **Eliminate Waste**: Identify and remove anything that does not add value to the customer. This includes unnecessary code, features, delays, and inefficiencies.
2. **Build Quality In**: Ensure quality at every step of the development process rather than relying solely on end-of-process testing. This includes practices like continuous integration, automated testing, and code reviews.
3. **Create Knowledge**: Foster a culture of learning and continuous improvement. Encourage experimentation, gather feedback, and use retrospectives to improve processes and practices.
4. **Defer Commitment**: Make decisions as late as possible to maintain flexibility and adapt to changes. This principle is also known as "decide as late as possible."
5. **Deliver Fast**: Shorten the delivery cycle to get feedback quickly and respond to customer needs promptly. This involves using iterative and incremental development techniques.
6. **Respect People**: Empower the team, involve everyone in decision-making, and encourage collaboration. Trust and respect for team members are crucial for motivation and productivity.
7. **Optimize the Whole**: Focus on the entire value stream and consider the impact of decisions on the whole system, not just individual parts. This means breaking down silos and improving cross-functional collaboration.

Examples in Practice:

- **Eliminate Waste**: Use tools and techniques like Value Stream Mapping to identify waste in the development process.
- **Build Quality In**: Implement Test-Driven Development (TDD) and Continuous Integration (CI) to ensure high-quality code.
- **Create Knowledge**: Conduct regular retrospectives and adopt a culture of continuous learning.
- **Defer Commitment**: Use Agile practices like Backlog Grooming to prioritize work closer to the time it will be done.
- **Deliver Fast**: Implement short iterations or sprints, and aim for continuous delivery.
- **Respect People**: Foster a collaborative environment with regular team meetings and open communication channels.
- **Optimize the Whole**: Look at the end-to-end process from idea to delivery and optimize the flow of work across the entire team or organization.

By adhering to these principles, Lean software development aims to create more efficient, effective, and adaptable software development processes.

Q. Is unit testing possible or even desirable in all circumstances? Provide examples to justify your answer.

No. If a module has 3 or 4 subordinates that supply data essential to a meaningful evaluation of the module, it may not be possible to conduct a unit test without "clustering" all of the modules as a unit.
The quality of the units is better and integrated quality of the system is also better. A bug discovered as of these first tests will further save much time in the development.

Unit testing is a valuable practice in software development, but it is not always possible or desirable in all circumstances. Here are some examples to illustrate when unit testing is appropriate and when it might not be:

When Unit Testing is Possible and Desirable:

1. **Core Business Logic**:
    - **Example**: In an e-commerce application, unit tests for calculating total order prices, applying discounts, or validating user input.
    - **Justification**: Core business logic is crucial and often changes. Unit tests ensure that changes do not introduce new bugs and help maintain a high level of reliability.
2. **Mathematical Algorithms**:
    - **Example**: Algorithms for financial calculations, data processing, or scientific computations.
    - **Justification**: These algorithms often have well-defined inputs and outputs. Unit tests can validate the correctness of the computations.
3. **Utility Functions**:
    - **Example**: Functions for string manipulation, date formatting, or file parsing.
    - **Justification**: Utility functions are typically small, pure functions with no side effects, making them ideal candidates for unit testing.

When Unit Testing is Challenging or Less Desirable:

1. **Code with Heavy Dependencies on External Systems**:
    - **Example**: Functions that rely heavily on databases, third-party APIs, or file systems.
    - **Justification**: Unit testing such code can be difficult because it often requires complex setup and teardown processes to simulate the external systems. Integration or end-to-end testing might be more appropriate.
2. **User Interface (UI) Components**:
    - **Example**: Visual components in a web application or mobile app.
    - **Justification**: UI components often change frequently, and their behavior can be tightly coupled with the look and feel of the application. Automated UI tests (using tools like Selenium or Appium) or manual testing might be more effective.
3. **Prototype or Experimental Code**:
    - **Example**: Code written for proof-of-concept projects or exploratory research.
    - **Justification**: The primary goal of this code is to test ideas quickly, and it may be thrown away or significantly refactored. Writing unit tests for such code can be seen as an unnecessary overhead.
4. **Highly Dynamic or Configurable Code**:
    - **Example**: Code that changes behavior based on configuration files or runtime parameters.
    - **Justification**: It can be challenging to cover all possible configurations with unit tests. In such cases, a combination of unit tests for common configurations and comprehensive integration tests might be more effective.

Balancing Unit Testing with Other Testing Approaches

While unit testing is an important part of the testing pyramid, it is not the only form of testing. A balanced testing strategy typically includes:

- **Integration Testing**: To ensure that different modules or services work together correctly.
- **System Testing**: To validate the behavior of the entire system against the requirements.
- **Acceptance Testing**: To verify that the system meets the needs of the end-users.
- **Exploratory Testing**: To uncover unexpected issues through manual testing.

Conclusion

Unit testing is a powerful technique that improves code quality and maintainability, but its applicability depends on the context. Developers should consider the nature of the code, the development phase, and the testing goals when deciding whether to write unit tests. In some cases, other forms of testing might be more appropriate to achieve the desired level of quality and reliability.

Extras

Problem: Is it always possible to develop a strategy for testing software that uses the sequence of testing steps described in? What possible complications might arise for embedded systems?

Testing Strategy
Consider the steps in testing strategy:
1. Unit Testing
2. Integration Testing
3. Validation Testing
4. System Testing

From the above testing steps, it is not possible to develop a strategy for testing embedded systems.
Because, while conducting through the unit testing, the complexity of a test environment is accomplished to the unit testing. It may not justify the benefit.
When these modules are felled behind the schedule in Integration, testing process is complicated by the scheduled availability. This situation occurs especially in the unit-tested modules
But, especially for embedded systems, the validation testing cannot be conducted outside of the target hardware configuration. For this reason, the validation and system testing are combined.
Hence, it is not always possible to develop software by supposed testing sequence.

Problem: Why is a highly coupled module difficult to unit test?

In highly coupled modules, errors can propagate to different module (tested module). So the paths need to be tested will increase due to the shared data or other module invocation. Highly coupled components are tightly connected to each other.
A highly coupled module is one who has at least part of its interface closely tied to one or more other modules. This means that the closely related modules can not be completely tested independently (nominal unit testing) but must be tested as a unit (more nearly sub-integrated testing). The larger (in terms of code size and complexity) the code, the more difficult it is to test. Further, action occurring in different modules is harder to trace, code read, design realistic tests for, etc.

How can project scheduling affect integration testing?

When the project scheduling is done, each module finished date will be established or set. In the other hand integration testing has to wait for all modules to finish and to be unit tested (if possible). So any changes to the project scheduling will directly affect the integration testing.

Because integration testing is done after a certain point in the project life time any changes to the schedule before that point will affect the integration test schedule.

Problem: Who should perform the validation test—the software developer or the software user? Justify your answer.

Validation testing is done in two phase: Alpha testing and beta testing.

Alpha testing is done in the controlled environment in the company by the users before release of the software whereas beta testing is done after the release of software in the market in uncontrolled environment by the software users.

Problem: Do you design software when you “write” a program? What makes software design different from coding?

No, writing a program is the different concept in design software. Design is the place where software quality is established. Before starting of design software, first requirements should be analyzed and specified.
In the software design process, design engineering is the one of the concept. While beginning software, requirements have been analyzed and modeled. This model can be accessed for quality and improved before code is generated.
In a software engineering context, first need to develop the models of program. Not the program themselves.
Software design different from coding:
At first it is very clear that, design is not coding and coding is not design. It is created from program components. Design is the description of the logic, which is used in solving the problem. Coding is the language specification which is implementation of the design. It runs on the computer and, provides the expected result.

Problem: If a software design is not a program (and it isn’t), then what is it?

Yes, software design is not a program. Coding or programming is a language which is used to represent the design. Programming is not good for representing details of architecture, components or their collaborations.
Software design contains a set of principles, concepts, and practices which leads to development of high quality product. Design plays an important role in the development of a successful software product.
The main aim of design is to create a model that presents firmness, commodity, and delight to customers those who use it. Software engineering continuously changes with new methods for better analysis and to evolve broader understanding.
Software design is like a kernel of software engineering. Software design is an action with the software engineering modelling activity, followed by code generation and testing.
The following are design methods which are implemented in software engineering:
• Object/Class design
• Architectural design
• Interface design
• Component design

Problem: How do we assess the quality of a software design?

The quality of software is assessed based on the design even before it is implemented. During design, the quality is assessed by conducting a series of technical reviews.
The following are the guidelines to assess the quality of the software design:
• A design should be simple created by using recognizable patterns which are easier for implementation and testing.
• A design should be modular that is, it can be broken down into smaller sub systems.
• A design should lead to data structures, components, and interfaces that are appropriate for classes and in turn reduce the complexity of connection with the external environment.
• A design is derived using repeatable methods and should be represented using simple notations which communicates its meaning.
The following are the FURPS quality attributes which are like a benchmark for a quality software design:
• Functionality

• Usability
• Reliability
• Performance
• Supportability

Problem: Examine the task set presented for design. Where is quality assessed within the task set? How is this accomplished? How are the quality attributes achieved?

Generic task set for design:
1. Examine information domain model and design appropriate data structures for data objects and their attributes
2. Select an architectural pattern appropriate to the software based on the analysis model
3. Partition the analysis model into design subsystems and allocate these subsystems within the architecture
4. Create a set of design classes or components
5. Design any interface required with external systems or devices
6. Design the user interface
7. Conduct component level design
8. Develop a deployment model
Quality is assessed for each element of the design model--data design, architecture, user interface, and component-level design
During design, assessing quality is accomplished by conducting a series of technical reviews (TRs). A technical review is a meeting conducted by members of the software team
Quality Attributes:

Quality Attributes represent a target for all software design. Set of attributes are represented as an acronym FURPS.
F – Functionality
U – Usability
R – Reliability
P – Performance
S – Supportability
These quality attributes can be achieved when these attributes are addressed before a design commences, but not after the design is complete and construction has begun.

Problem: Provide examples of three data abstractions and the procedural abstractions that can be used to manipulate them.

Data abstractions: It is the process of representing essential details not including internal details.
Examples:
1) First example of data abstraction is to abstract “Doors” as a data structure with essential properties:
Door: Properties
Type
Swing direction,
Manufacturer,
Insets,
Lights,
Weight,
Opening / Close mechanisms
2) Second example of data abstraction is “College ”
College: Computer science,

Electrical Engineering
Mechanical Engineering
3) Third example of data abstraction is “Software company:
Company: Software Architect
Software Manager
Sr. Developers
Jr. Developers
Tester
Procedural abstractions: The Procedural abstraction is to decompose problem to many simple sub works.
Examples:
1) Credit Card: Validate(User name, Password)
Check limited (Withdrawal limit)
2) Website: Add page(linking new page to home page)
Access (setting access permissions)
3) Online Medical transitions : patient list
Cost (Medical Lab bills)

Problem: Describe software architecture in your own words.

The software architecture is the structures of program components / modules. And
Software architecture is:
• An overall view of the solution to a problem
• The high-level design of modular components and how they interact
• A foundation that one can build on to solve a problem (e.g., rules, policies, attributes, etc.)
• An efficient method to meet a fixed set of well-defined attributes
Architecture needs to include:
architecture: The architectural vision, style, principle, key communication and control mechanisms and concepts that guide the team of architects in the creation of the architecture.
Architectural patterns: Structural patterns such as layers and client/server and mechanisms such as brokers and bridges.
Architectural views: Structural views help document and communicate the architecture in terms of the components and their relationships and are useful in assessing architectural qualities like extensibility and behavioral views are useful in thinking through how the components interact to accomplish their assigned responsibilities and evaluating the impact of what if scenarios on the architecture. Behavioral views are especially useful in assessing run time qualities such as performance and security. Execution views help in evaluating physical distribution options and documenting decisions.
Component specification: components are identified and assigned responsibilities that client components access through "contracted" interfaces. Component interconnections specify communication and control mechanisms and allow component interactions to accomplish system behavior. And key architectural design principles including abstraction, separation of concerns, postponing decisions, and simplicity, and related techniques such as interface hiding and encapsulation, as well as system decomposition principles and good interface design.

Problem: Some people say that “variation control is the heart of quality control.” Since every program that is created is different from every other program, what are the variations that we look for and how do we control them?

A program is equal to data structure plus algorithm. Generally, different programmers will design a program in a different way. Logic of one person varies from the other as per their own thinking. Coding of one programmer will be different from others in solving the same problem.

So, we can expect variations in the design and coding of the data structure and the algorithm of a program by different individuals. Also the programming language chosen by each may vary. Hence we can look for the variations also in the syntax, logic, complexity, readability.
We can control the variations by checking the types and levels of complexity that can be used in the design of the data structures and the algorithms of a program.

Problem: Is it possible to assess the quality of software if the customer keeps changing what it is supposed to do?

Yes, If we define quality as "conformance to requirements," and requirements are dynamic (keep changing), it is possible to assess the quality of software. The definition of quality will be dynamic and an assessment of quality will be difficult

Problem: Quality and reliability are related concepts but are fundamentally different in a number of ways. Discuss the differences.

Quality and reliability both are related concepts. But fundamentally these both are different in some scenarios.
1. Quality control is concerned with the performance of a product at one point in time.
2. Quality focuses on the software's conformance to explicit and implicit requirements.
3. Software quality assurance is the mapping of the managerial principles and design disciplines of quality assurance onto the applicable managerial and technological space of software engineering.
Where as
1. Reliability is concerned with the performance of a product over its entire life time.
2. Reliability focuses on the ability of software to function correctly as a function of time or some other quantity.
3. Software reliability models extend measurements, enabling collected defect data to be extrapolated into projected failure rates and reliability predictions.

Problem: Can a program be correct and still not be reliable? Explain.

Reliability: Yes, it is possible for a program to conform all explicit functional and performance requirements at a given instant. Because, reliability uses statistical analysis to determine the software failure with likelihood occur. And, The probability of failure is free software operation for a specified period of time in a specified environment.

Problem: Why is there often tension between a software engineering group and an independent software quality assurance group? Is this healthy?

Software Quality Assurance group vs. Software Engineering group:
Between these two groups, there is a common natural tension because, if the SQA group takes on the role of the "observing the bugs of software code," flagging quality problems and high-lighting shortcomings in the developed software, this is normal and this would not be embraced with open arms by the software engineering group.
This software quality assurance group helps ensure that they fit the project’s needs and verifies that they will be usable for performing the necessary reviews and audits throughout the software life cycle.
As long as the tension does not degenerate into hostility, there is no problem. It is important to note, however, that a software engineering organization should work to eliminate this tension by encouraging a team approach that has development and QA people working together toward common goal of high quality software.

Problem: You have been given the responsibility for improving the quality of software across your organization. What is the first thing that you should do? What’s next?

For improving the quality of software across the organization, First of all i evaluate the quality Based on the formal technical reviews, I checkout the total working process of that software. If it is working smoothly, then the number of SQA activities might be implemented. That is
Change control and SCM;
Comprehensive testing methodology;
SQA audits of documentation and related software.
In this process software metrics can help.

Problem: Besides counting errors and defects, are there other countable characteristics of software that imply quality? What are they and can they be measured directly?

Suppose maintainability measured by mean – time – to – change; portability as measured by an index that indicates conformance to language standard, portability complexity, availability, reliability.
Portability as measured by an index that indicates conformance to language standard; complexity as measured by McCabe's metric, and so on
Here reliability focuses on the ability of software to function correctly as a function of time or some other quantity.
Safety considers the risks associated with failure of a computer based system that is controlled by software.
Quality focuses on the software's conformance to explicit and implicit requirements.
In most of the cases an assessment of quality considers may factors that are qualitative in nature.